

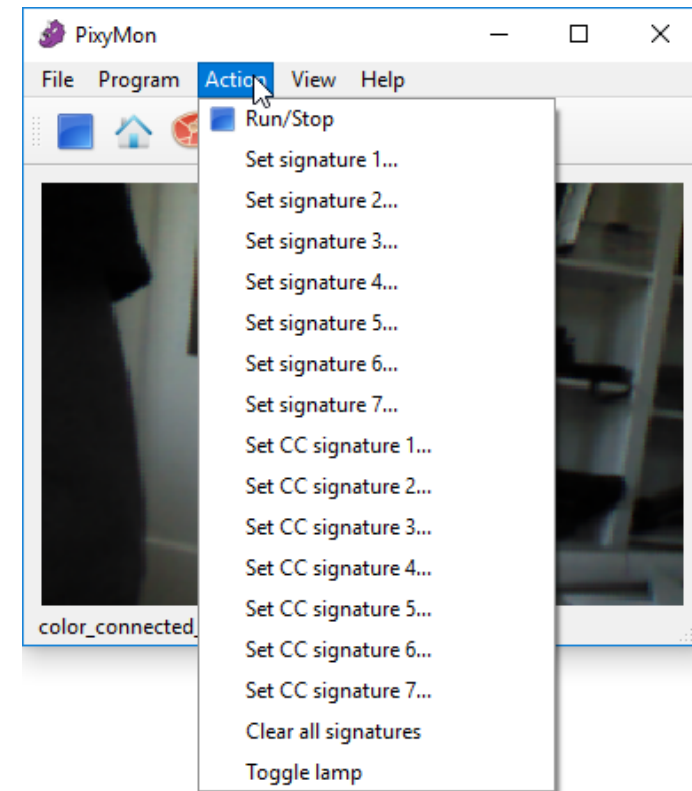
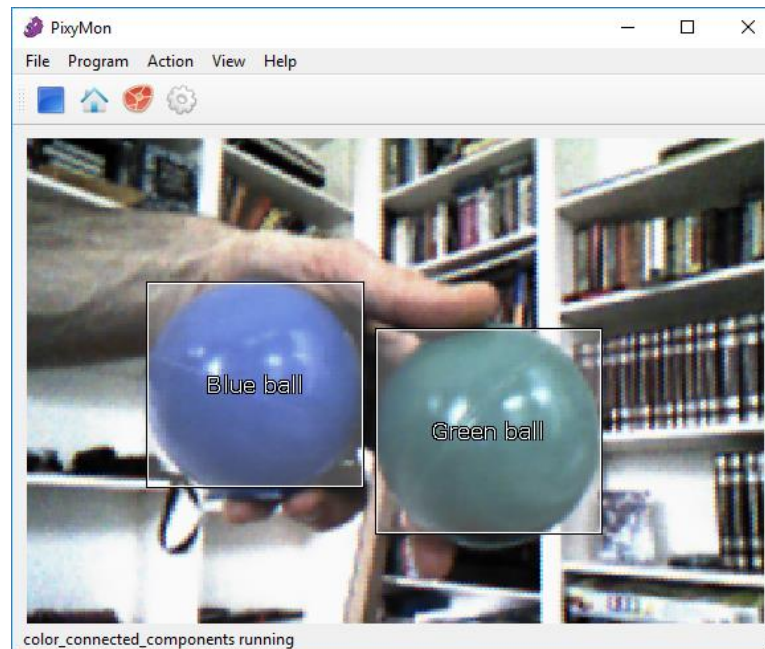
เริ่มต้นใช้งานกล่อง Pixy



ใช้จำแนกสีได้ตามที่เรากำหนด



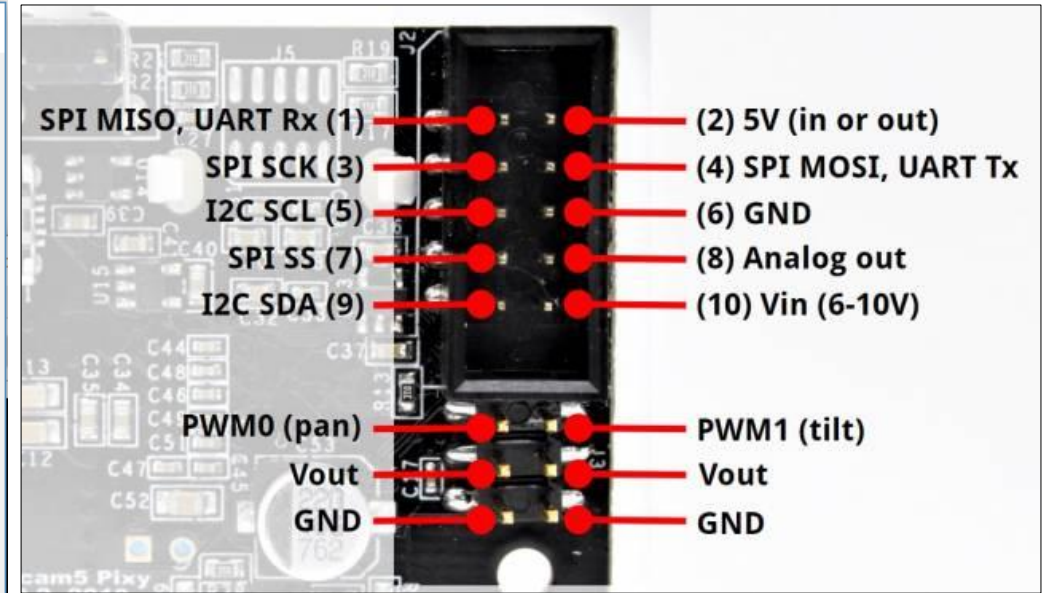
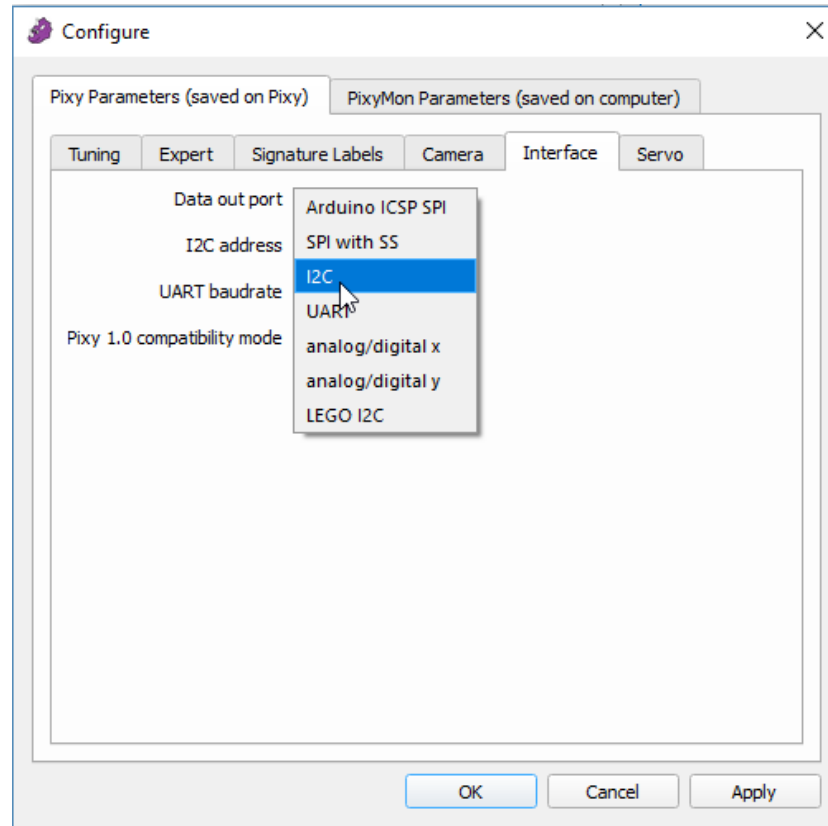
Pixy



เชื่อมต่อกับบอร์ดไมโครคอนโทรลเลอร์ด้วย SPI , IIC หรือ UATR



Pixy



ติดตั้งโปรแกรม PixyMon v2

PixyMon V2 คือ โปรแกรมช่วยตั้งค่าต่างๆของกล้อง Pixy

ดาวน์โหลดได้ที่ <https://pixycam.com/downloads-pixy2>

PixyMon v2

PixyMon v2 is the configuration utility for Pixy2 that runs on Windows, MacOS and Linux.

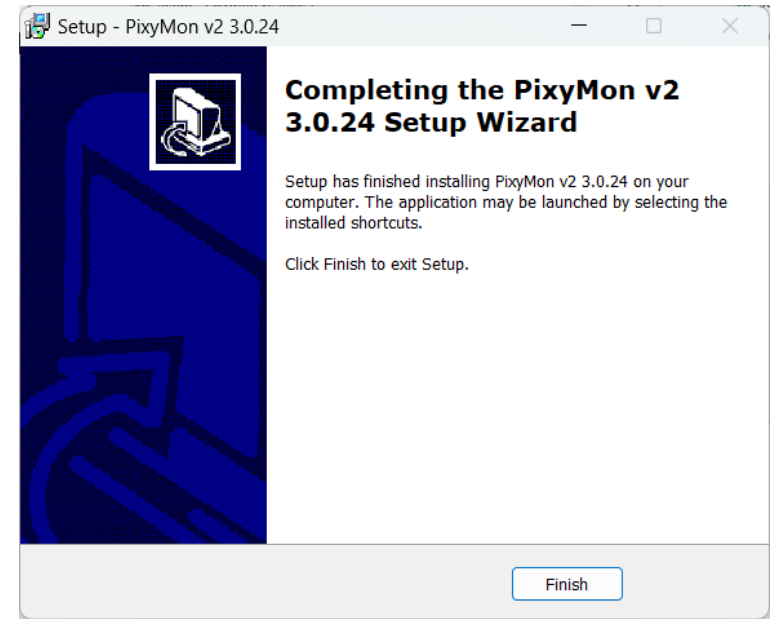
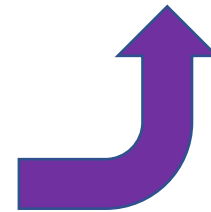
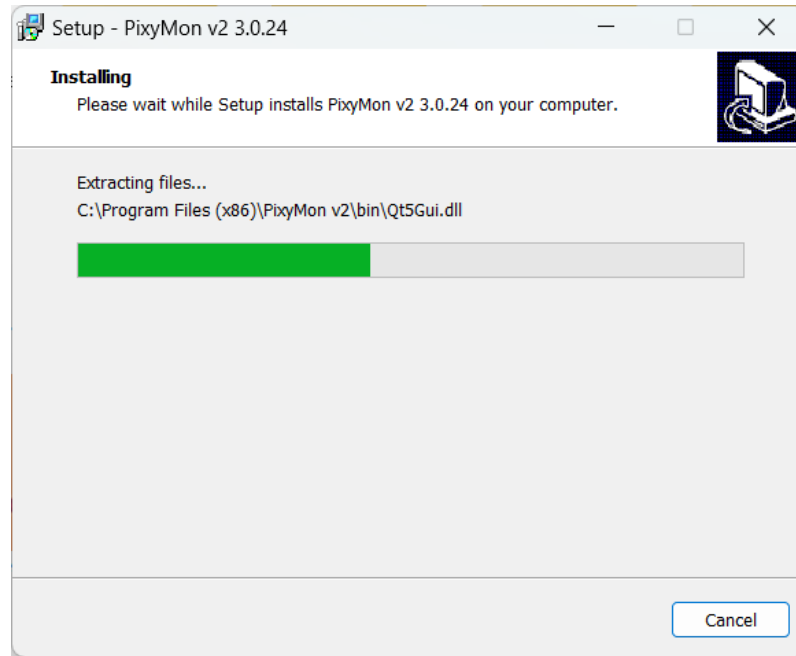
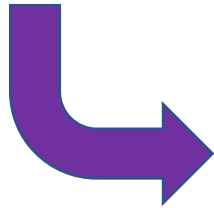
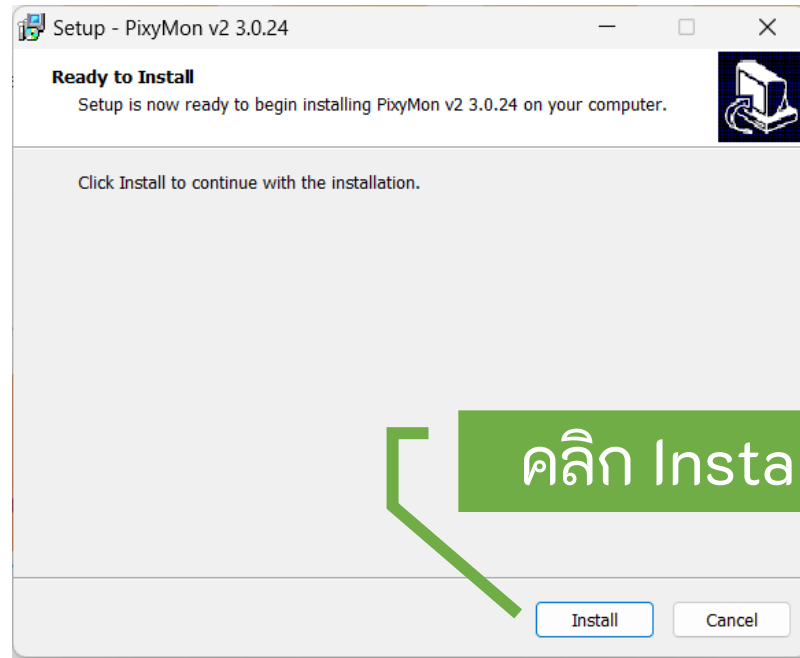
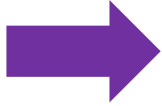
- [Pixymon v2 Windows version 3.0.24 \(exe\)](#)
 - [installation docs for Windows Vista, 7, 8, 10](#)
 - [installation docs for XP](#)
- [PixyMon v2 Mac version 3.0.24 \(dmg, High Sierra\)](#)
 - [installation docs](#)
- [Linux Pixymon v2 is available through github](#)
 - [installation docs](#)

เลือกติดตั้งให้ตรงตามระบบปฏิบัติการของเครื่องคอมพิวเตอร์

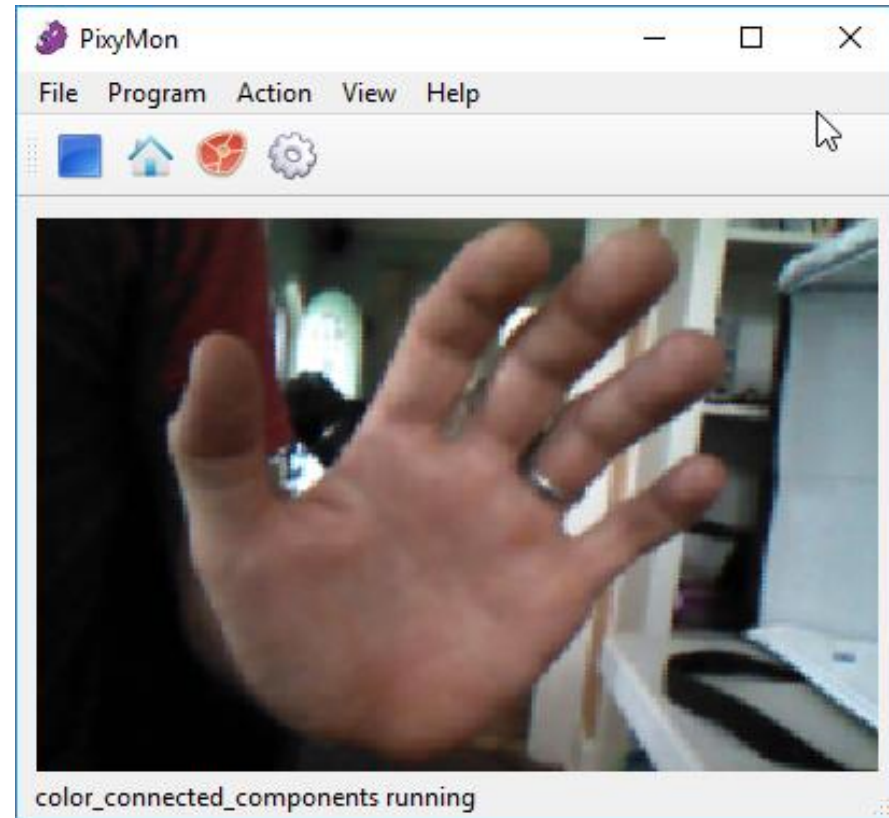
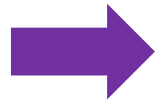
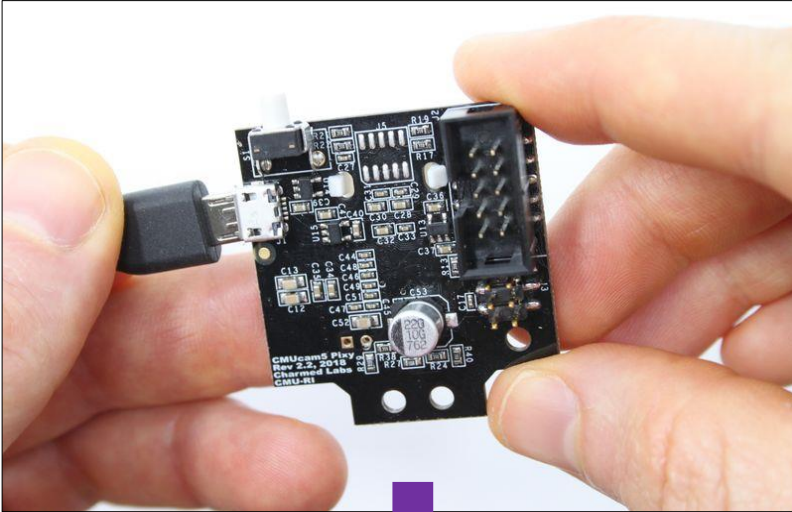
ดับเบิลคลิกเพื่อติดตั้ง



pixymon_v2_windo
ws-3.0.24.exe



ทดสอบเชื่อมต่อ Pixy เข้ากับคอมพิวเตอร์



เปิดโปรแกรม PixyMon เพื่อทดสอบ โปรแกรมจะแสดง
ภาพจากกล้องขึ้นมาโดยอัตโนมัติ

อัปเดตเฟิร์มแวร์

ดาวน์โหลดได้ที่

<https://pixycam.com/downloads-pixy2>

Pixy2 firmware

Pixy2 firmware is code that runs on Pixy2 itself.

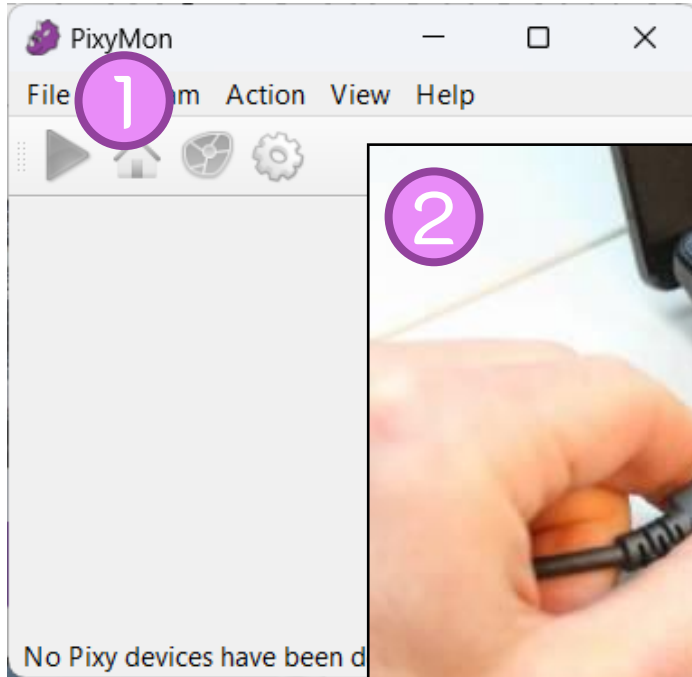
- Pixy2 general firmware version 3.0.18 (hex)
- Pixy2 LEGO firmware version 3.0.18 (hex)
 - installation docs



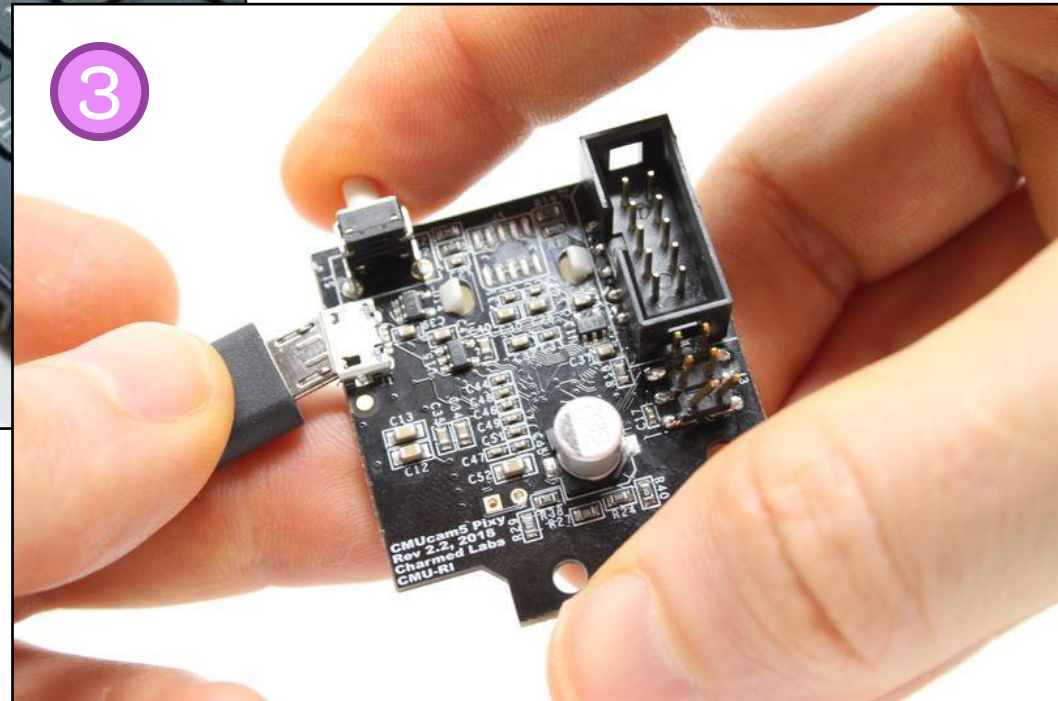
pixy2_firmware-3.0.18-g
eneral.hex



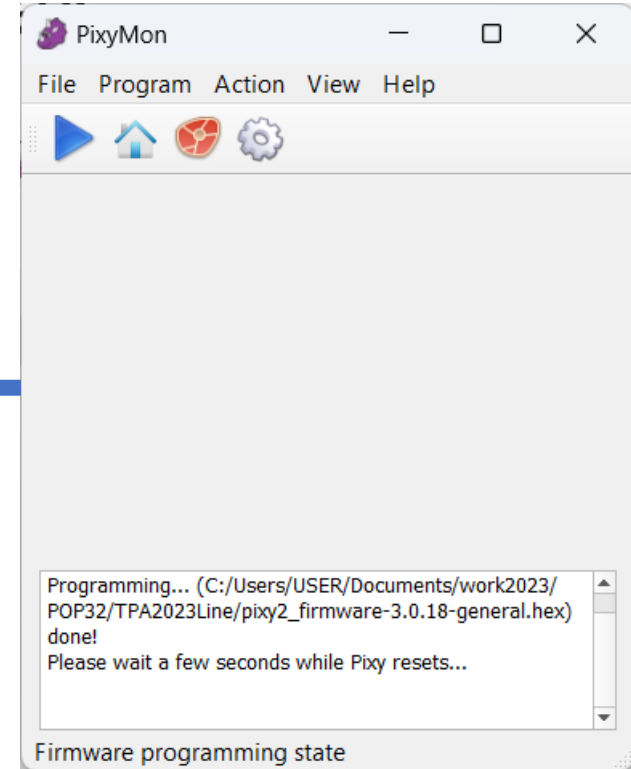
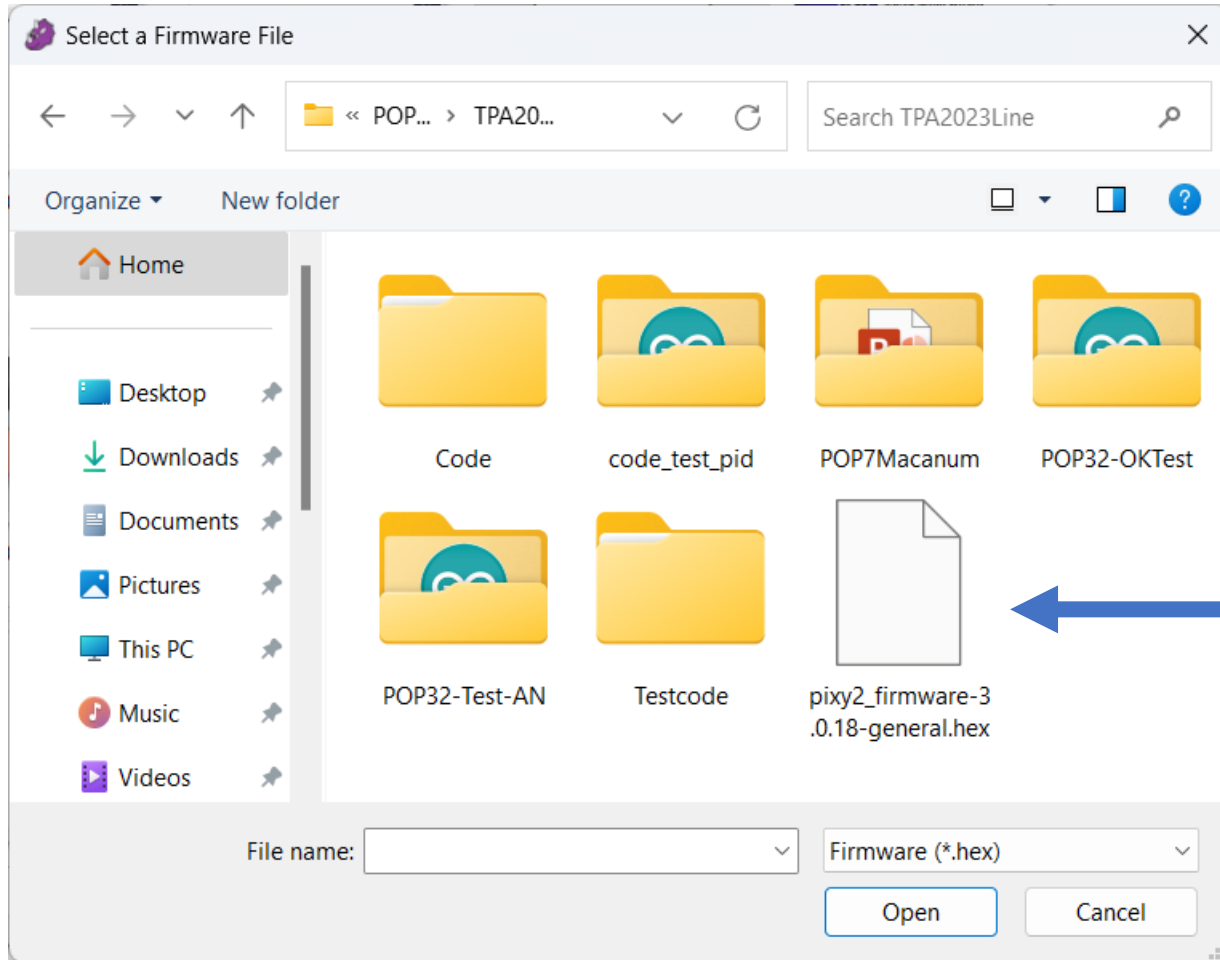
อัปเดตเฟิร์มแวร์



กดปุ่ม S1 ก่อนแล้วค่อยเสียบ USB



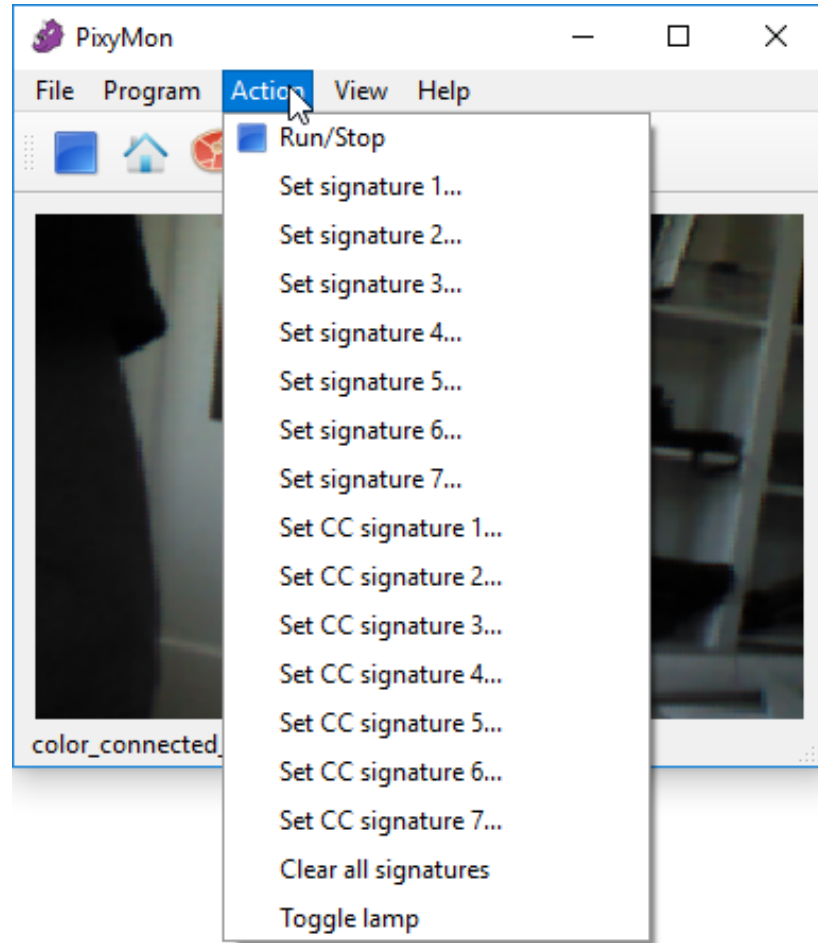
เลือกไฟล์ .hex



เมื่อเลือกเสร็จแล้ว

โปรแกรมจะเปิดหน้าต่างขึ้นมาให้เลือกไฟล์

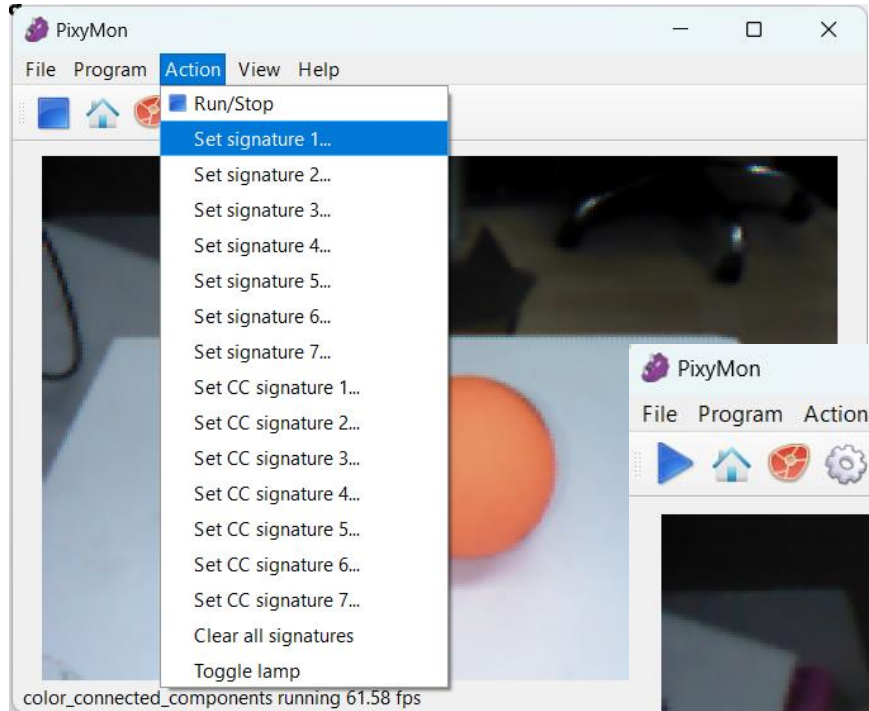
การตั้งค่าเพื่อจำแนกสี



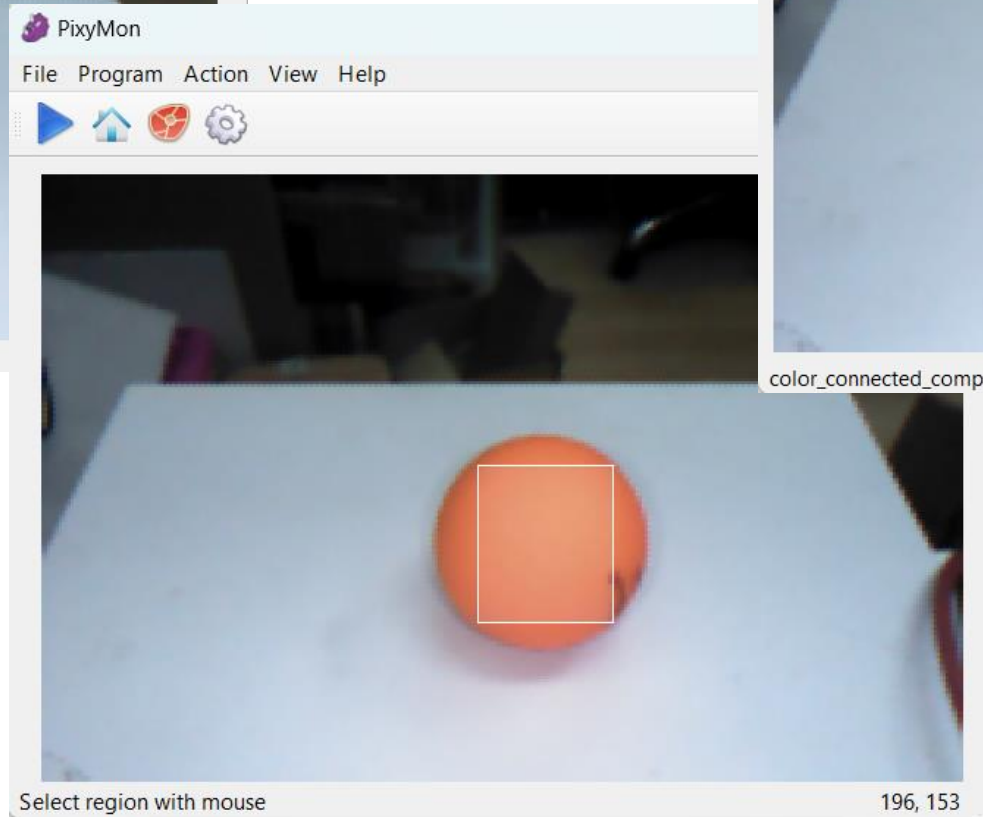
ตัวเลือกเมนู

1. Run/Stop : สั่งให้หยุดการทำงานหรือทำงาน
2. Set signature : การจำแนกแบบสีเดียว
3. Set CC signature : การจำแนกแบบรหัสสี
4. Clear all signatures : ล้างการจำแนกทั้งหมด
5. Toggle lamp : เปิด-ปิด LED บน Pixy

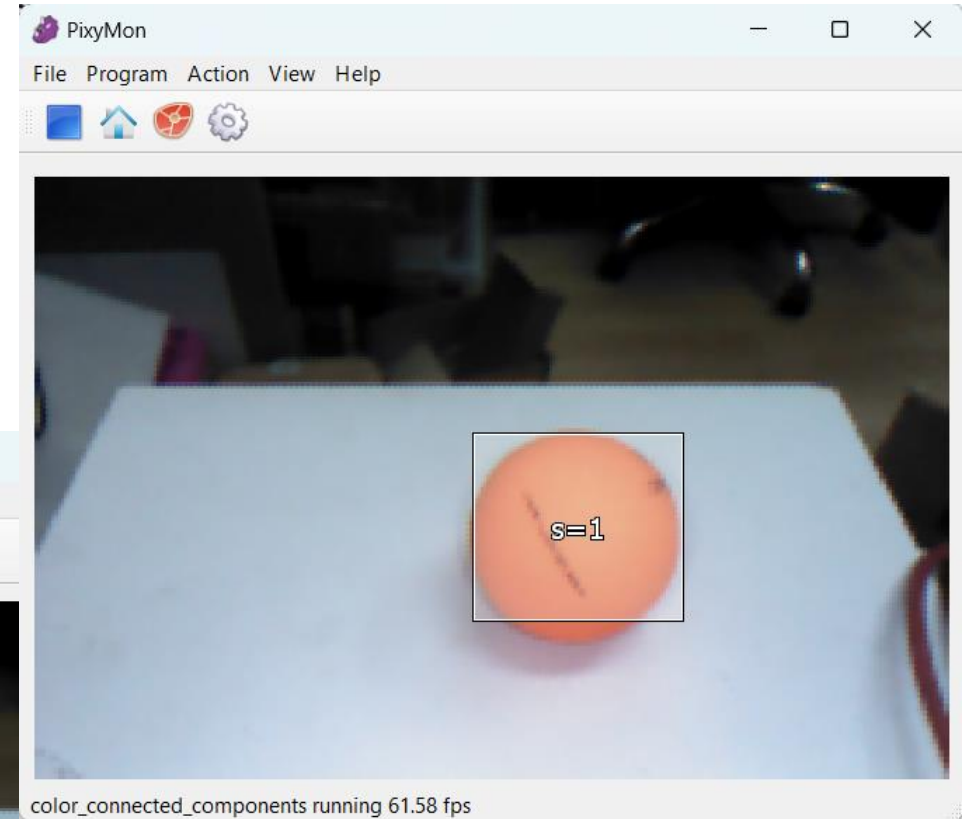
เลือกการจำแนกแบบสีเดียว



เลือกหมายเลข



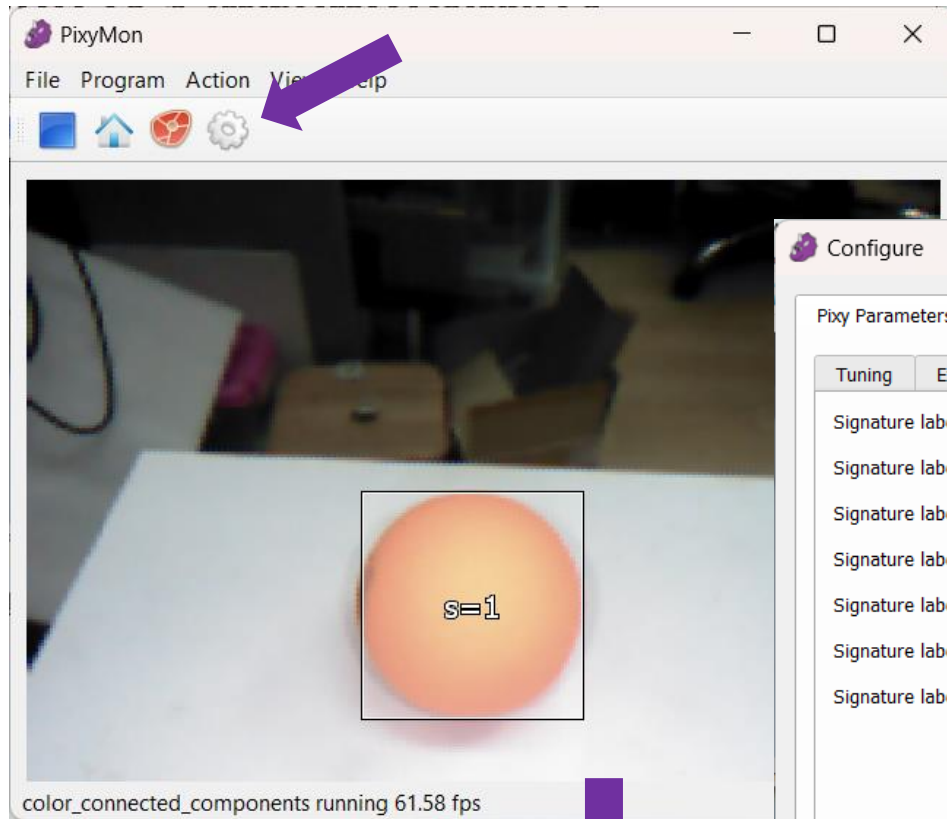
เลือกสี



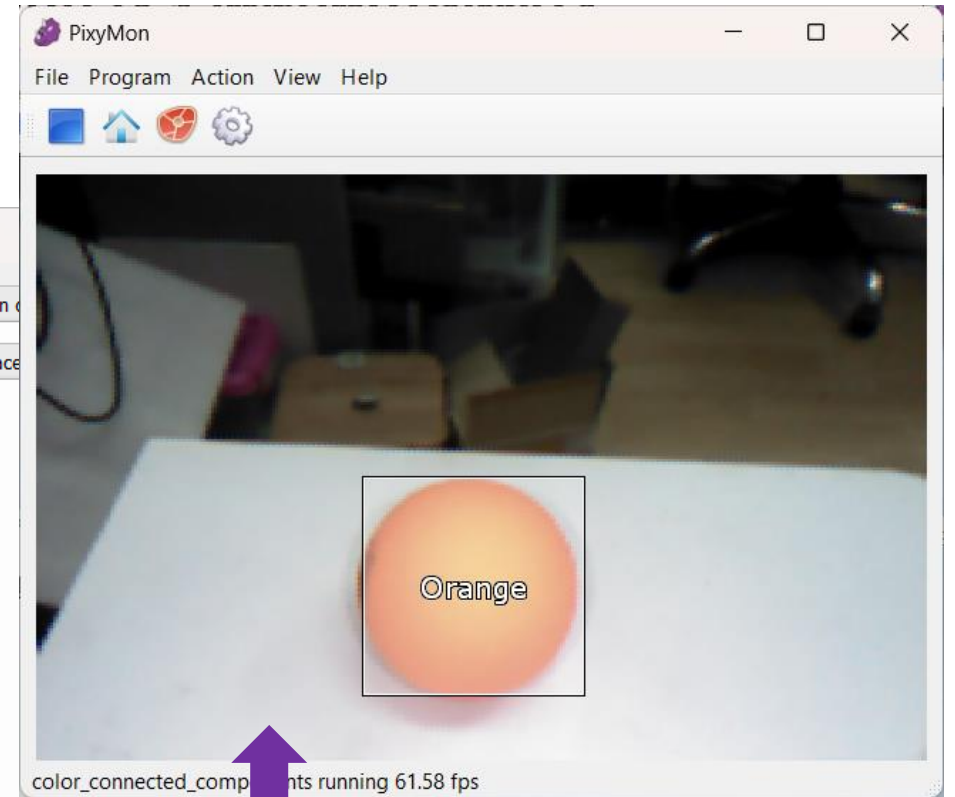
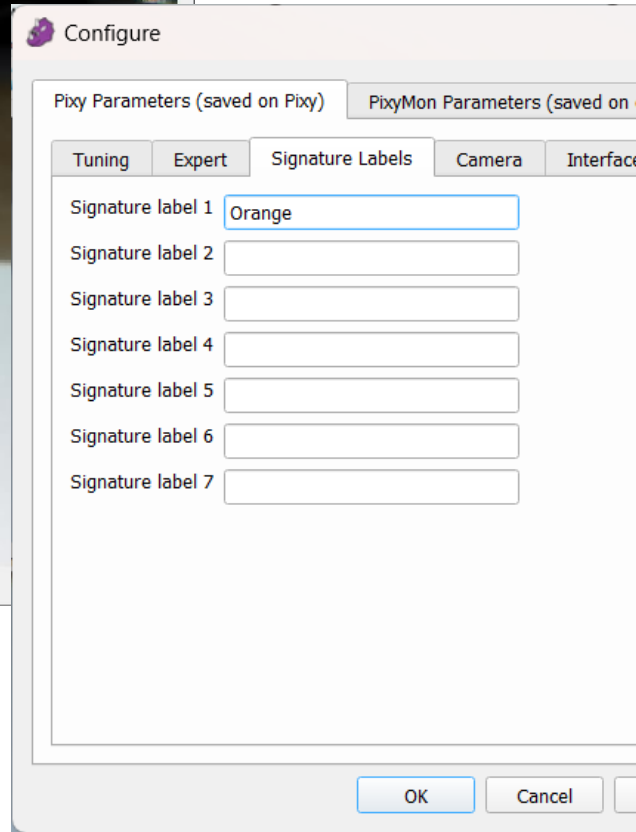
ผลลัพธ์



ตั้งชื่อวัตถุสีใหม่



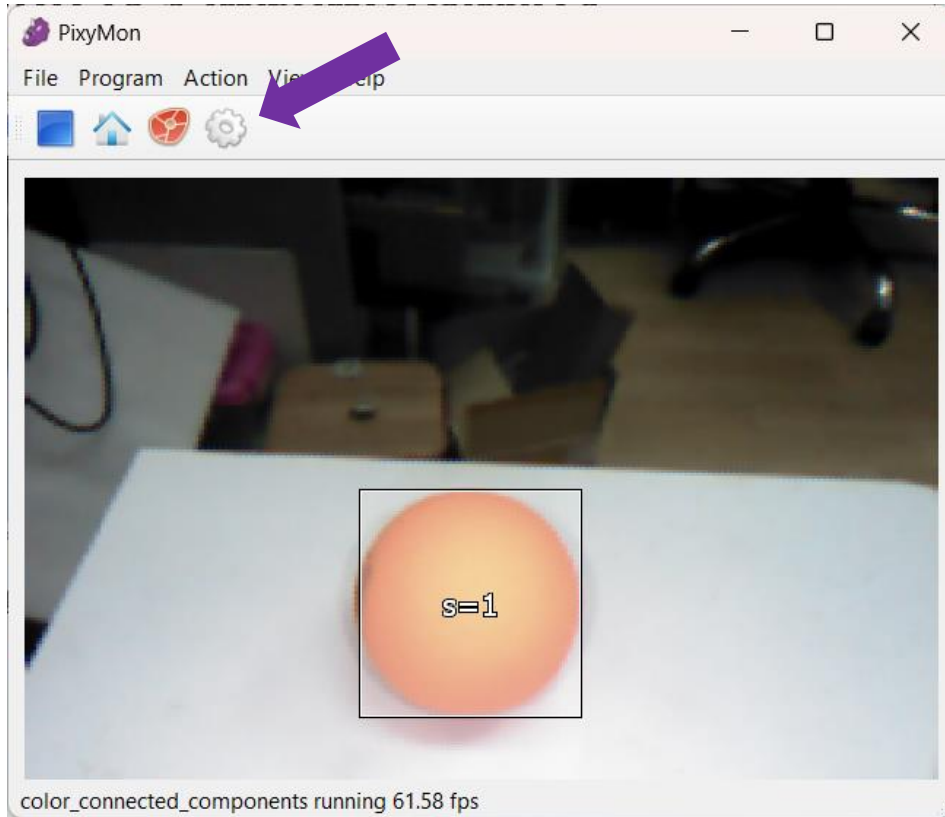
เลือกตั้งค่า



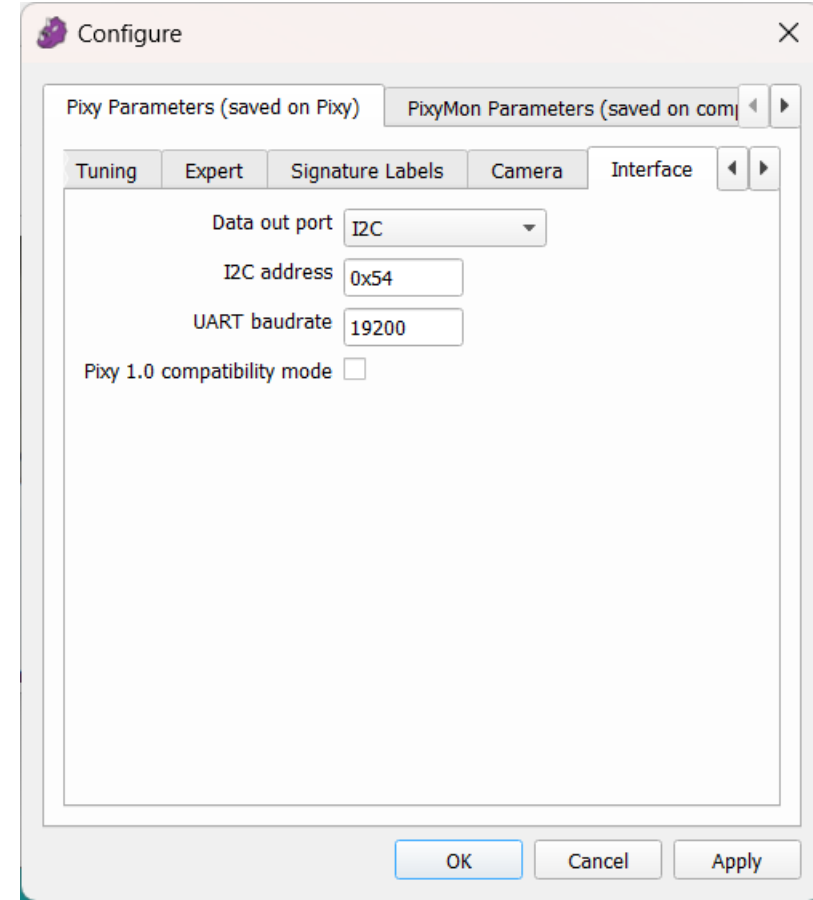
ผลลัพธ์

เลือก Signature Labels เปลี่ยนชื่อแล้วกดปุ่ม OK

กำหนดรูปแบบการเชื่อมต่อ

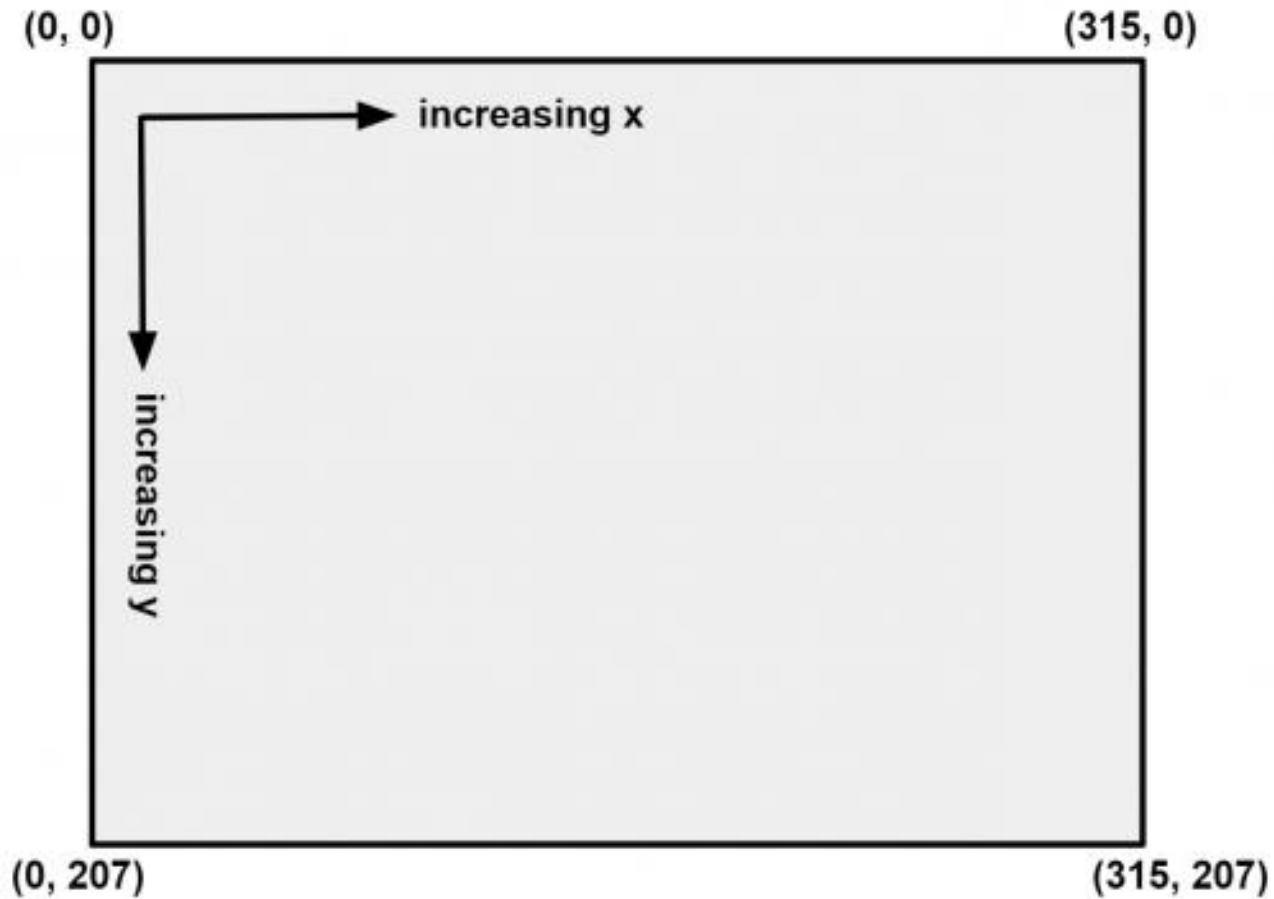


เลือกตั้งค่า



เลือก Interface เปลี่ยน Data out port เป็น I2C แล้วกดปุ่ม OK

พิกัดภาพจากมุมมองของ Pixy2

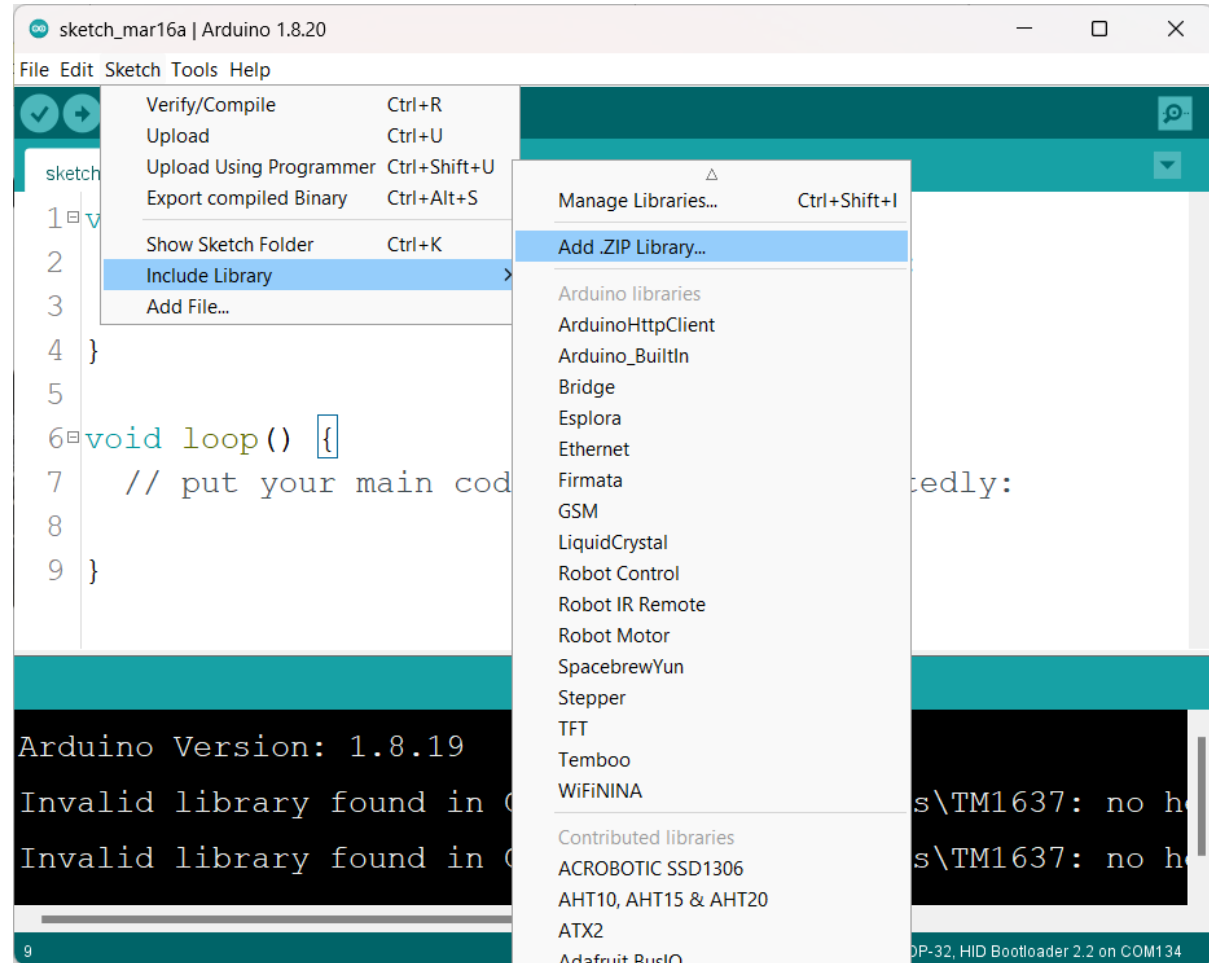


ติดตั้งไลบรารีสำหรับ POP-32

ดาวน์โหลดไลบรารีได้ที่ https://inex.co.th/store/software/POP32_Pixy2.zip

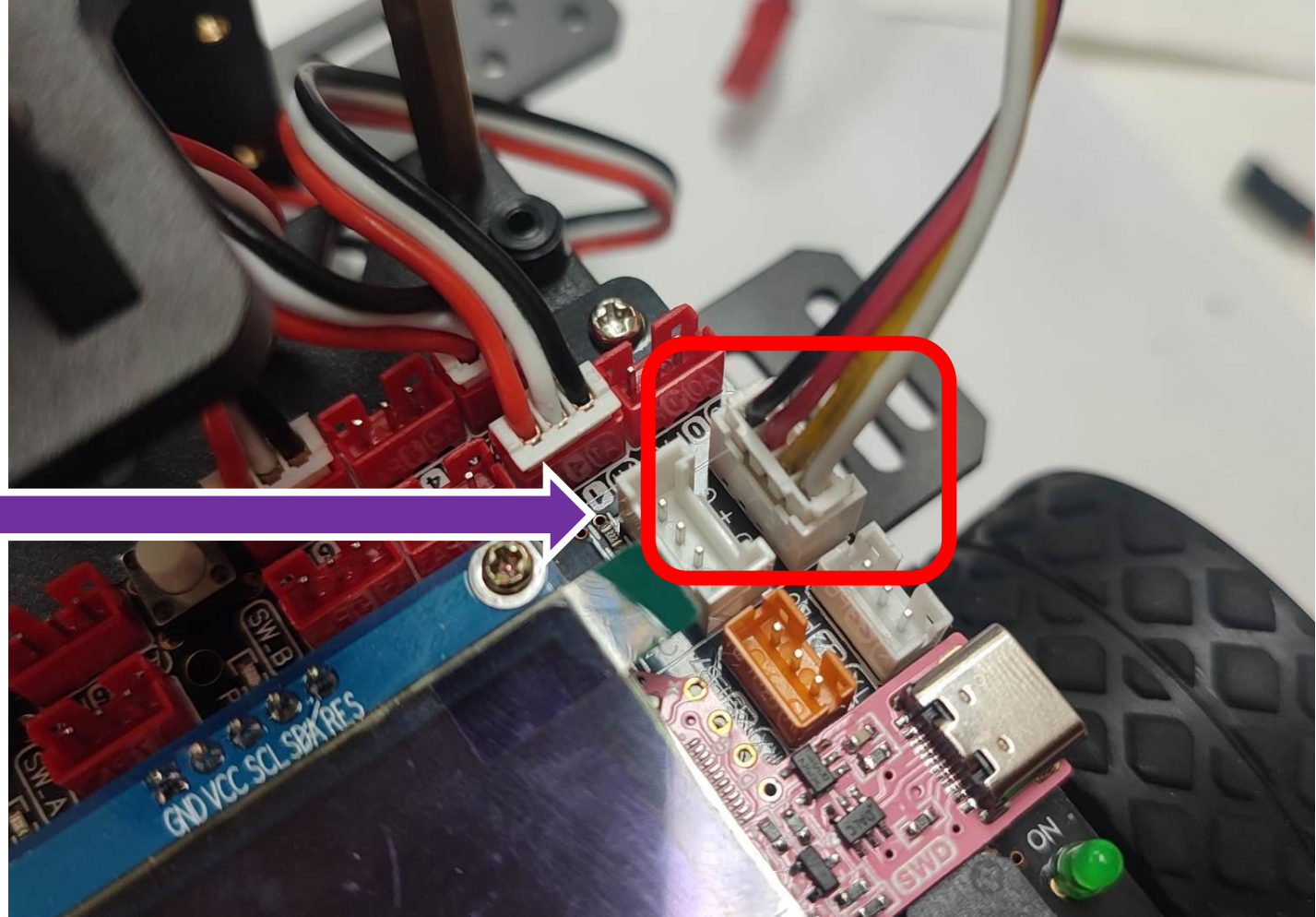
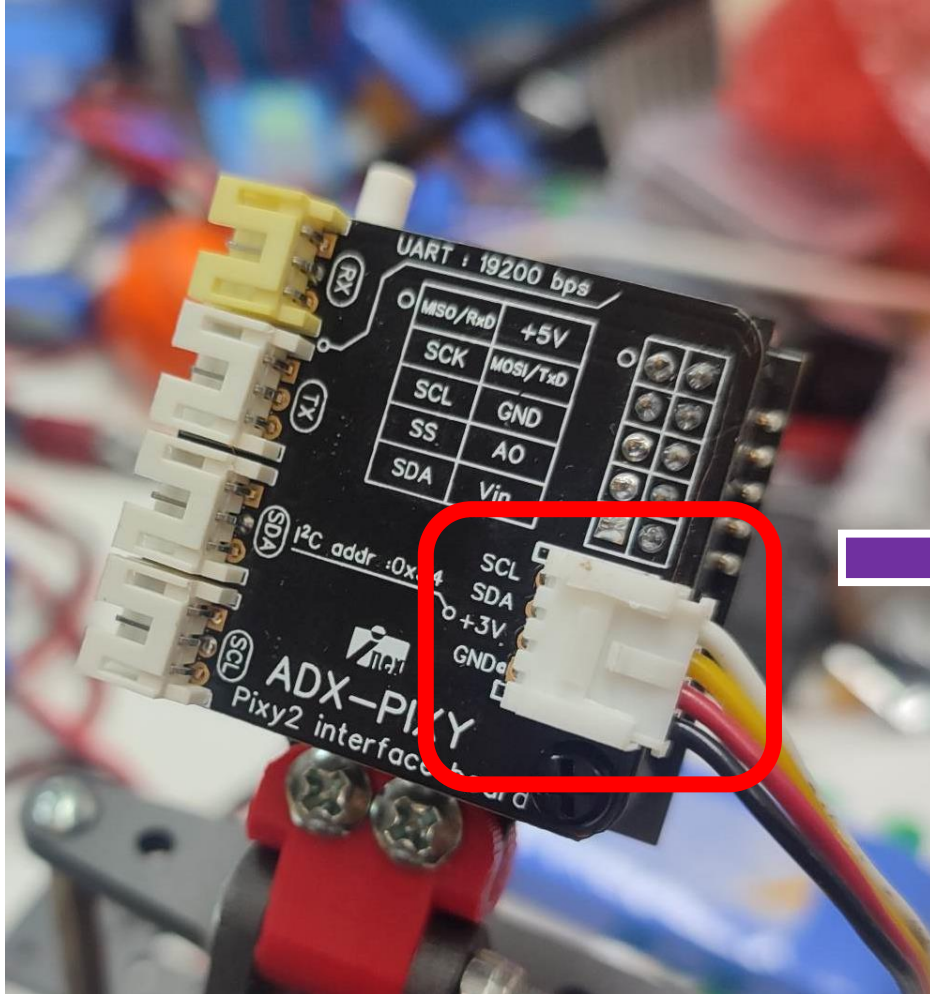


POP32_Pixy2.zip



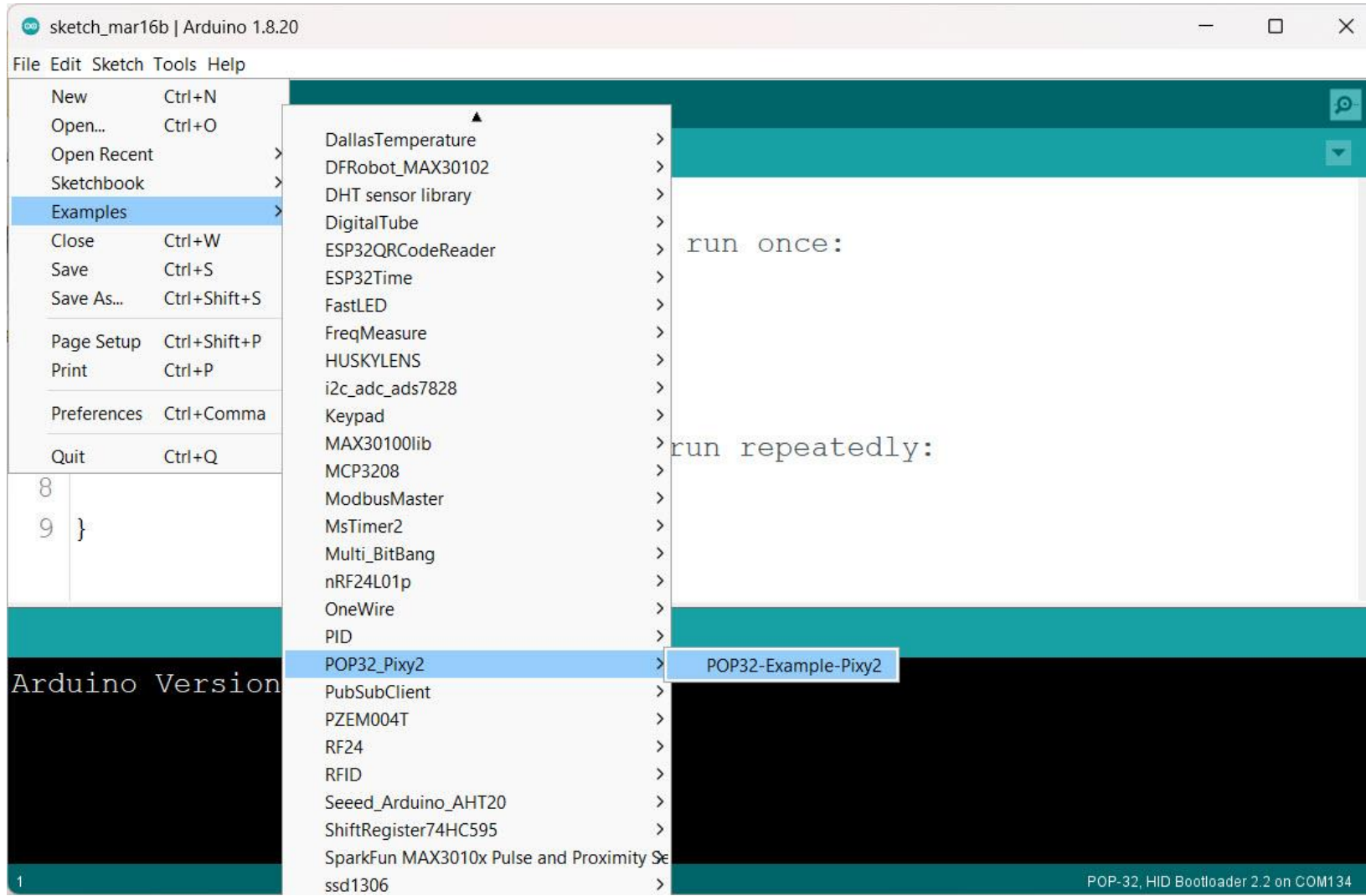
เข้าเมนู Sketch -> Include Library -> Add.Zip Library.. จากนั้นเลือกไฟล์ POP32_Pixy2.zip

ติดตั้ง Pixy เข้ากับบอร์ด POP-32



ใช้การเชื่อมต่อแบบ I2C

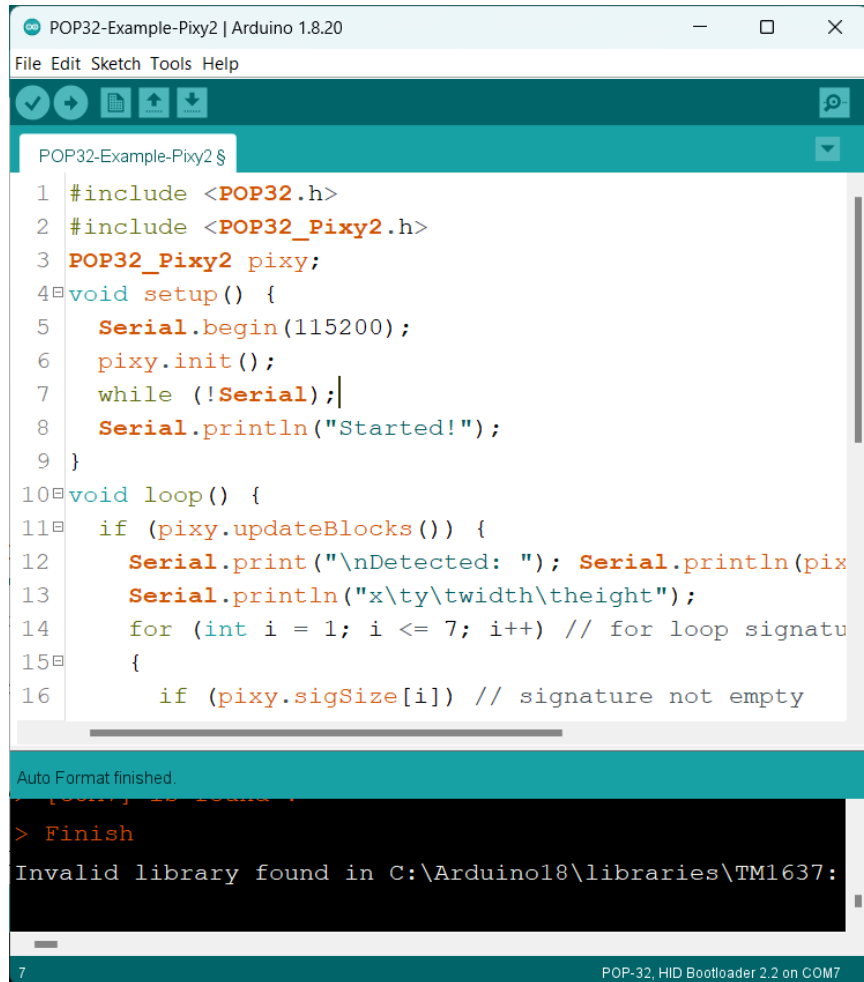
ทดสอบไลบรารีสำหรับ POP-32



ไปที่ File -> Examples -> POP32_Pixy2 -> POP32-Example-Pixy2

ทดสอบไลบรารีสำหรับ POP-32

Upload โปรแกรมส่งบอร์ดเพื่อทดสอบ



```
POP32-Example-Pixy2 | Arduino 1.8.20
File Edit Sketch Tools Help

POP32-Example-Pixy2 $
1 #include <POP32.h>
2 #include <POP32_Pixy2.h>
3 POP32_Pixy2 pixy;
4 void setup() {
5     Serial.begin(115200);
6     pixy.init();
7     while (!Serial);
8     Serial.println("Started!");
9 }
10 void loop() {
11     if (pixy.updateBlocks()) {
12         Serial.print("\nDetected: "); Serial.println(pixy.getDetectedCount());
13         Serial.println("x\ty\twidth\theight");
14         for (int i = 1; i <= 7; i++) // for loop signature
15         {
16             if (pixy.sigSize[i]) // signature not empty
17                 Serial.print(i);
18         }
19     }
20 }
```

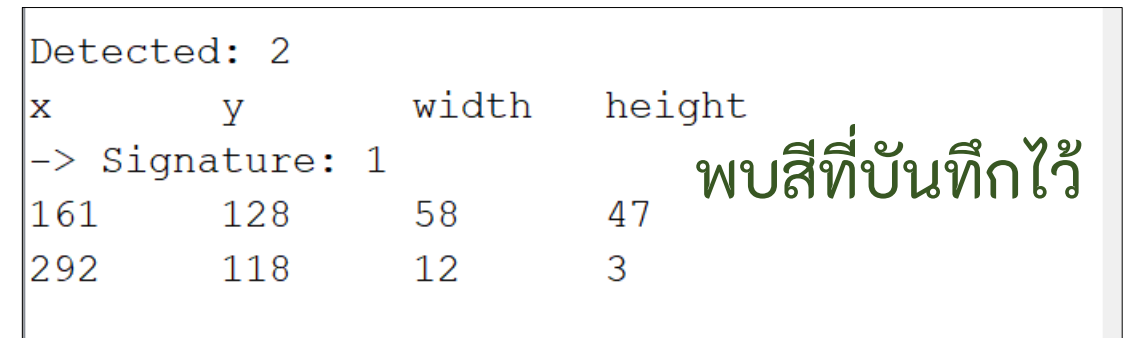
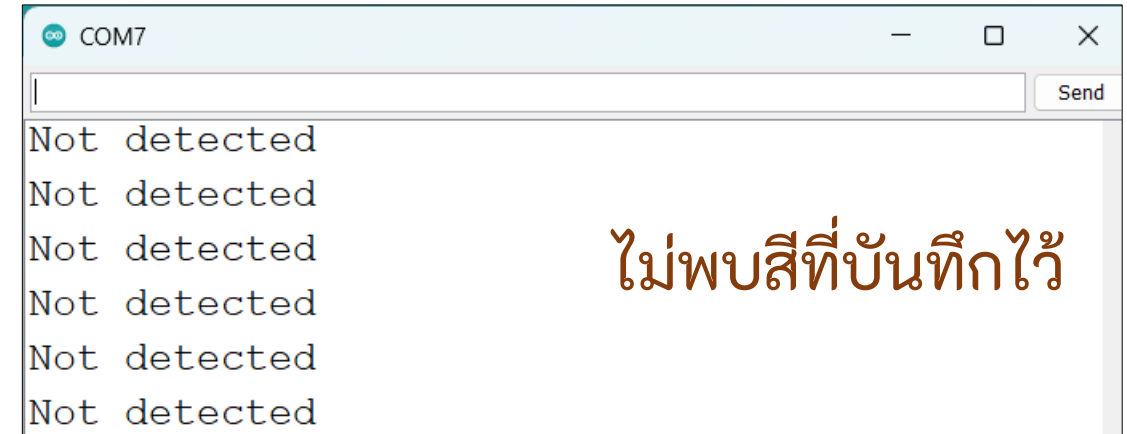
Auto Format finished.

> Finish

Invalid library found in C:\Arduino18\libraries\TM1637:

POP-32, HID Bootloader 2.2 on COM7

เปิด Serial Monitor



อธิบายคำสั่งไลบรารีของ Pixy

`#include <POP32_Pixy2.h>` ผนวกไฟล์ไลบรารี

`POP32_Pixy2 pixy` กำหนดตัวแปรที่ชื่อว่า pixy มารองรับคำสั่งทั้งหมด

`pixy.init()` เริ่มต้นใช้งานกล่อง pixy

`pixy.updateBlocks()` ตรวจสอบบล็อกทั้งหมด

`pixy.getNumBlocks()` จำนวนบล็อกทั้งหมด

`pixy.sigSize[i]` นับจำนวนบล็อกทั้งหมดของแต่ละลักษณะ $i = 1$ ถึง 7

อธิบายคำสั่งไลบรารีของ Pixy

i = คุณลักษณะ 1 ถึง 7

j = ลำดับบล็อกในคุณลักษณะเดียวกัน

`pixy.sigSize[i]` นับจำนวนบล็อกทั้งหมดของแต่ละคุณลักษณะ

`pixy.sigInfo[i][j].x` ตำแหน่ง x ตรงกลางบล็อก

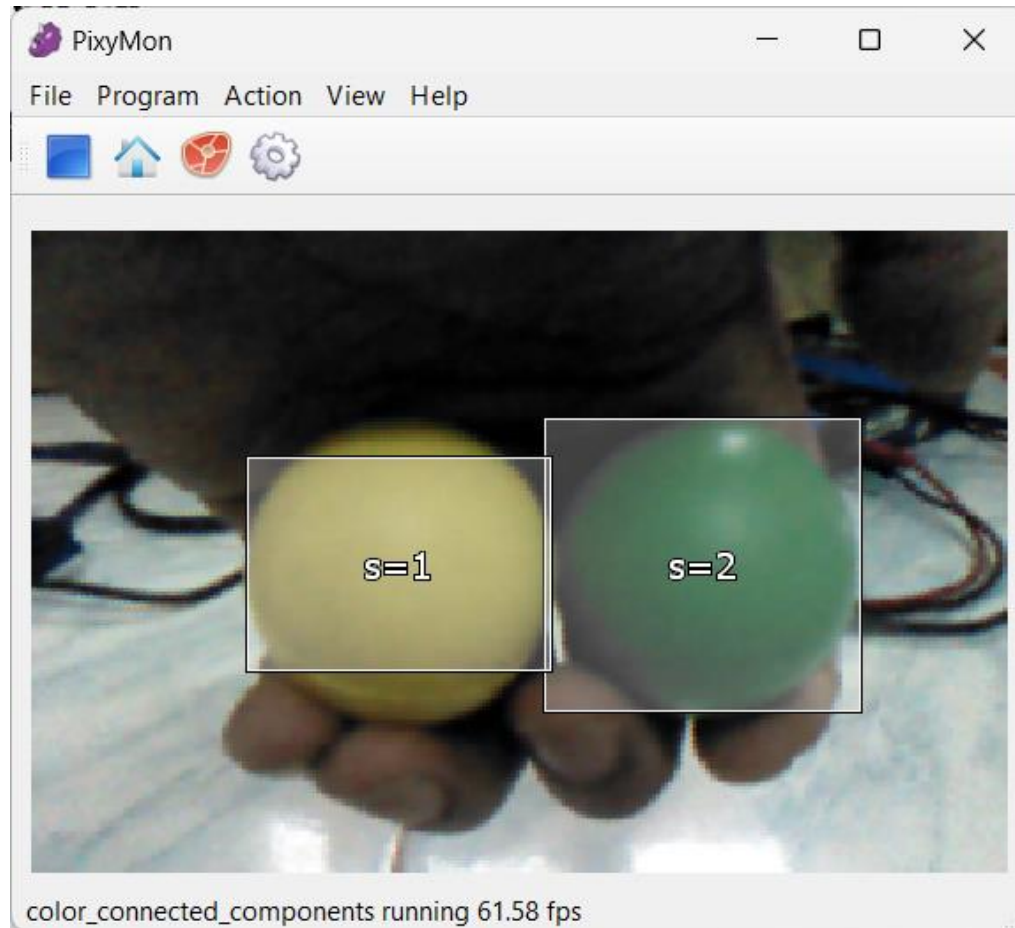
`pixy.sigInfo[i][j].y` ตำแหน่ง y ตรงกลางบล็อก

`pixy.sigInfo[i][j].width` ความกว้างของบล็อก

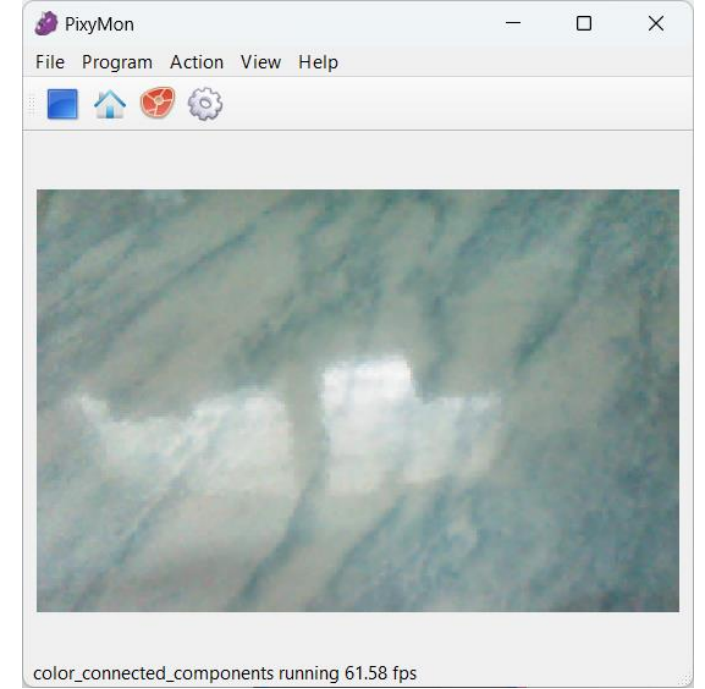
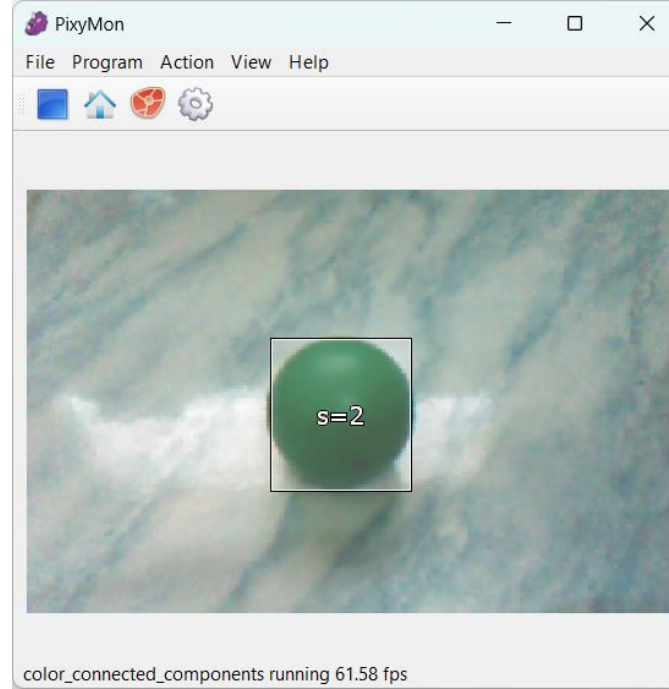
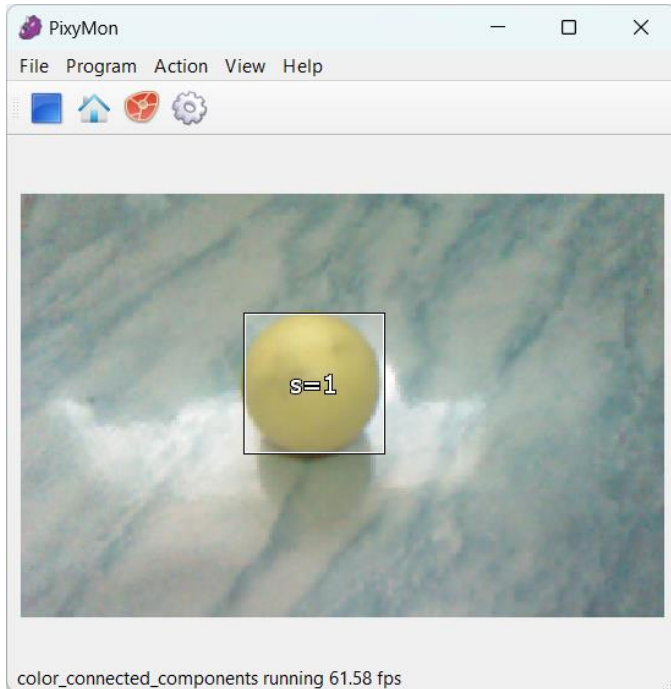
`pixy.sigInfo[i][j].height` ความสูงของบล็อก

การใช้ pixy แยกสี

นำลูกบอลสีมากกว่า 1 สีมาทำการกำหนดคุณลักษณะ



ผลลัพธ์ pixy แยกสี



ชุดคำสั่งตรวจสอบหมายเลขของคุณลักษณะ

```
for (int i = 1; i <= 2; i++){  
    if (pixy.sigSize[i]){  
        oled.text(0,0,"S=%d",i);  
        oled.show();  
    }  
}
```

ใช้คำสั่ง **for** เพื่อวนตรวจสอบ โดยใช้ **pixy.sigSize** เป็นเงื่อนไขในการตรวจพบ

โปรแกรมการใช้ pixy แยกสี

```
#include <POP32.h>
#include <POP32_Pixy2.h>
POP32_Pixy2 pixy;
void setup() {
  pixy.init();
  waitAnykey_bmp();
}
```

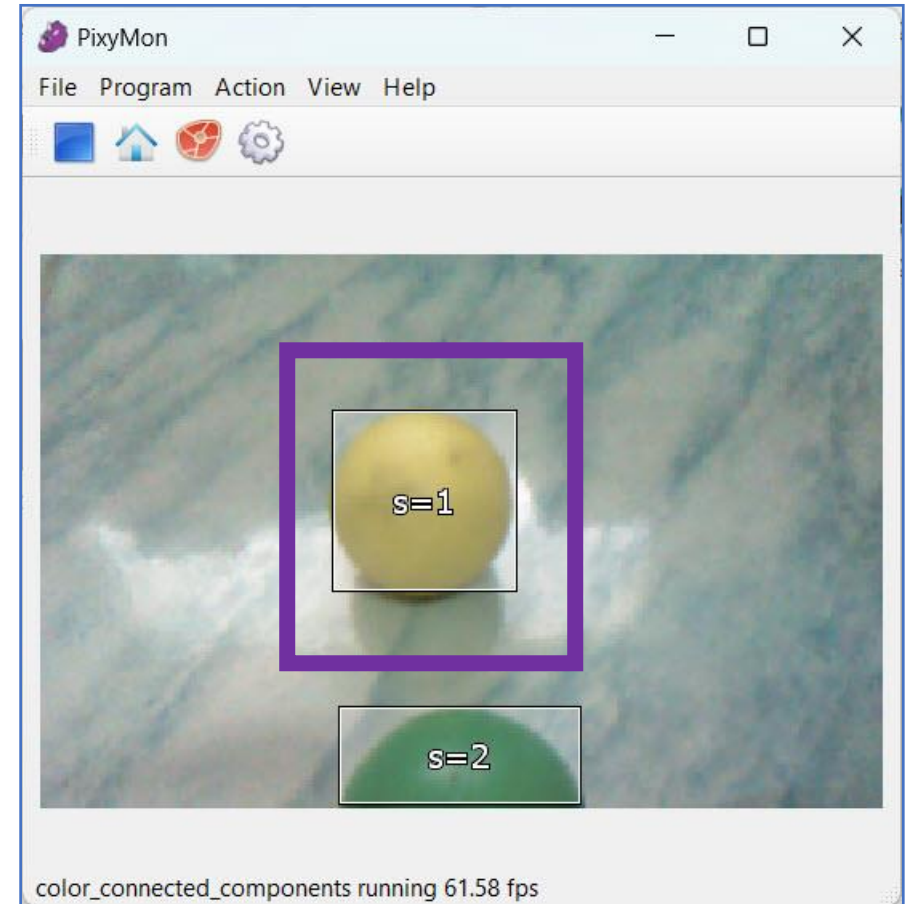
เริ่มต้นใช้งานกล่อง pixy และรอการกดปุ่ม

```
void loop() {
  if (pixy.updateBlocks()) {
    for (int i = 1; i <= 2; i++){
      if (pixy.sigSize[i]){
        oled.text(0,0,"S=%d",i);
        oled.show();
      }
    }
  }
  else{
    oled.text(0,0,"Not detected");
    oled.show();
  }
}
```

อ่านค่าและตรวจสอบแสดงออกหน้าจอ

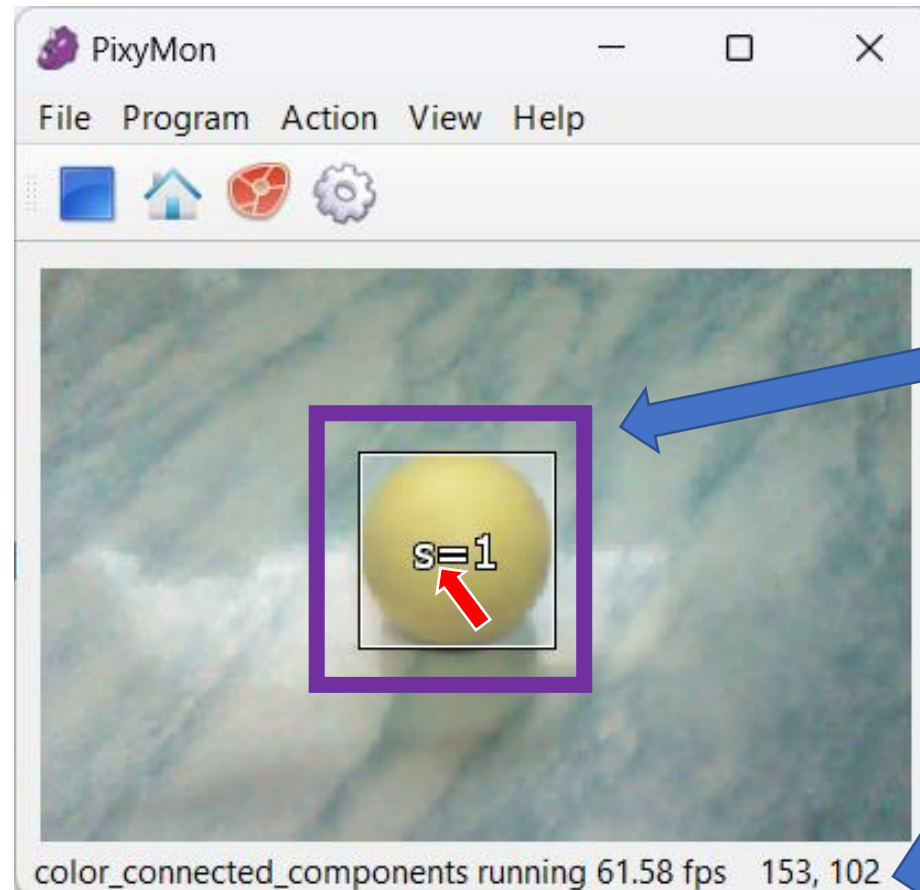
การใช้ pixy แยกสีแบบกำหนดพื้นที่ที่ตรวจจับ

จากภาพจะเห็นว่าวัตถุ 2 สีปรากฏบนกล้อง ทำให้การแยกสีไม่สามารถที่จะบอกได้ว่าเป็นสีที่เราต้องการหรือไม่ จึงจำเป็นต้องสร้างกรอบพื้นที่ขึ้นมาเพื่อใช้ตรวจสอบ
โดยใช้ พิกัด x,y ของวัตถุเข้ามาเป็นเงื่อนไข



กำหนดพื้นที่ที่ตรวจจับ

ต้องทราบขอบเขตพื้นที่ ที่ต้องการ ด้วยการชี้เมาส์ไปวางไว้ที่จุดพื้นที่ ที่เราต้องการ จากภาพจุดที่ต้องการคือ $x = 153, y = 102$ หรือกำหนดกรอบพื้นที่ตามความต้องการ



ขนาดกรอบที่ต้องการ
50x50

$x = 153, y = 102$

คำสั่ง

abs(value)

เป็นคำสั่งที่ใช้กำหนดค่าให้อยู่ในช่วงบวกเท่านั้น

ตัวอย่าง

```
int y = abs(-50)
```

y จะมีค่าเท่ากับ 50

โปรแกรมการใช้ pixy แยกสีแบบกำหนดพื้นที่ที่ตรวจจับ

```
int x = pixy.sigInfo[i][j].x;  
x = x-153;  
int y = pixy.sigInfo[i][j].y;  
y = y-102;
```

คำนวณหาค่าความผิดพลาดจากพิกัด x,y

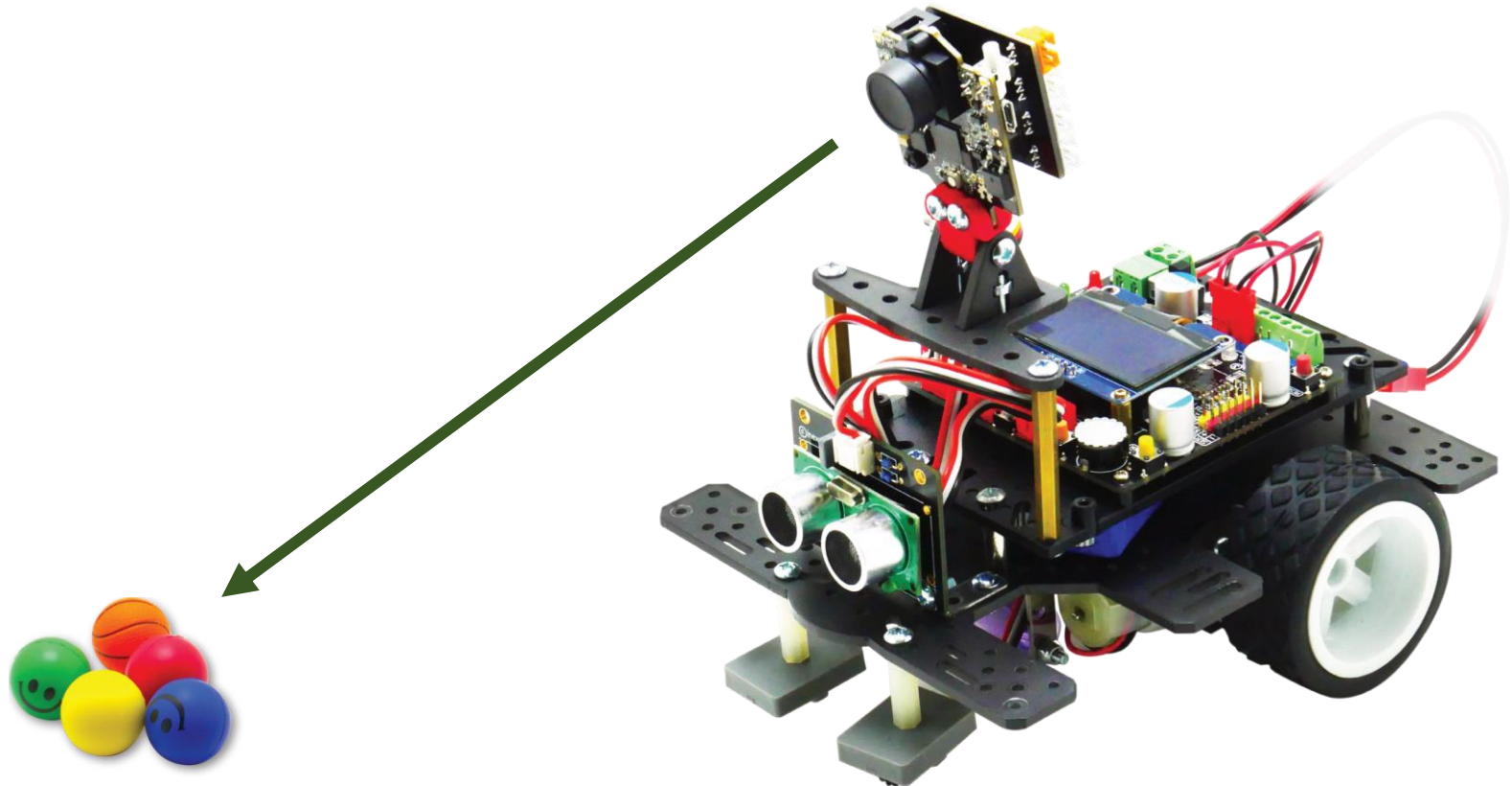
```
if((abs(x)<25)&&(abs(y)<25))  
{.....เมื่อตรวจพบในพื้นที่.....}  
else  
{.....เมื่อไม่พบในพื้นที่.....}
```

นำค่าความผิดพลาดมาเป็นเงื่อนไข

```
void loop() {  
  if (pixy.updateBlocks()) {  
    for (int i = 1; i <= 2; i++) {  
      if (pixy.sigSize[i]) { // signature not empty  
        for (int j = 0; j < pixy.sigSize[i]; j++) {  
          int x = pixy.sigInfo[i][j].x; x = x-153;  
          int y = pixy.sigInfo[i][j].y; y = y-102;  
          if((abs(x)<25)&&(abs(y)<25)){  
            oled.text(0, 0, "S=%d      ", i); oled.show();  
          }else{  
            oled.text(0, 0, "Not detected"); oled.show();  
          }  
        }  
      }  
    }  
    else {  
      oled.text(0, 0, "Not detected");  
      oled.show();  
    }  
  }  
}
```

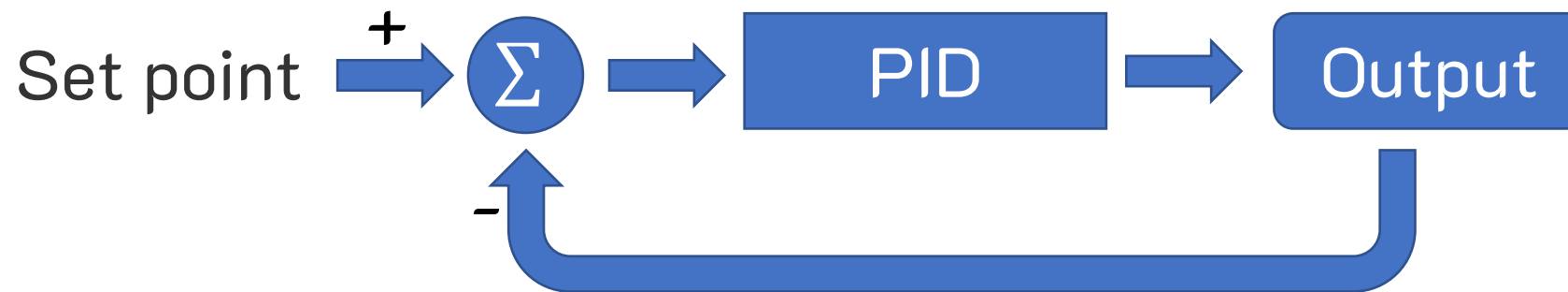
การเคลื่อนที่ติดตามลูกบอล

การเคลื่อนที่ติดตามลูกบอลจะต้องอาศัยการควบคุมความเร็วมอเตอร์ โดยที่ความเร็วมอเตอร์จะแปรผันตามระยะทางของลูกบอลอย่างรวดเร็วมีฉะนั้น จะทำให้ลูกบอลหลุดจากมุมมองกล้องได้



การควบคุมแบบ PID : Proportional Integral Derivative

เป็นการควบคุมแบบป้อนกลับ โดยใช้วิธีการนำค่าความผิดพลาดที่ได้จากผลต่างระหว่าง ค่าที่ต้องการ (Set point) กับค่าป้อนกลับมาคำนวณ เพื่อจะพยายามลดค่าความผิดพลาดให้เหลือน้อยที่สุด หรือให้ค่าความผิดพลาดเข้าใกล้ 0 ให้ได้มากที่สุด



การควบคุมแบบ PID : Proportional Integral Derivative

P : สัดส่วนความผิดพลาด

$$\text{Error} = \text{Set_point} - \text{feedback}$$

I : สัดส่วนความผิดพลาดสะสม

$$\text{SumError} = \text{SumError} + \text{Error}$$

D : สัดส่วนความผิดพลาดปัจจุบันต่ออดีต

$$\text{dError} = \text{Error} - \text{PrevError}$$

สมการ PID จะนำค่า k_p, k_i, k_d มาเพิ่มอัตราการขยายในแต่ละเทอม

$$\text{Output} = (\text{Error} * k_p) + (\text{SumError} * k_i) + (\text{dError} * k_d)$$

หน้าที่ของแต่ละเทอม

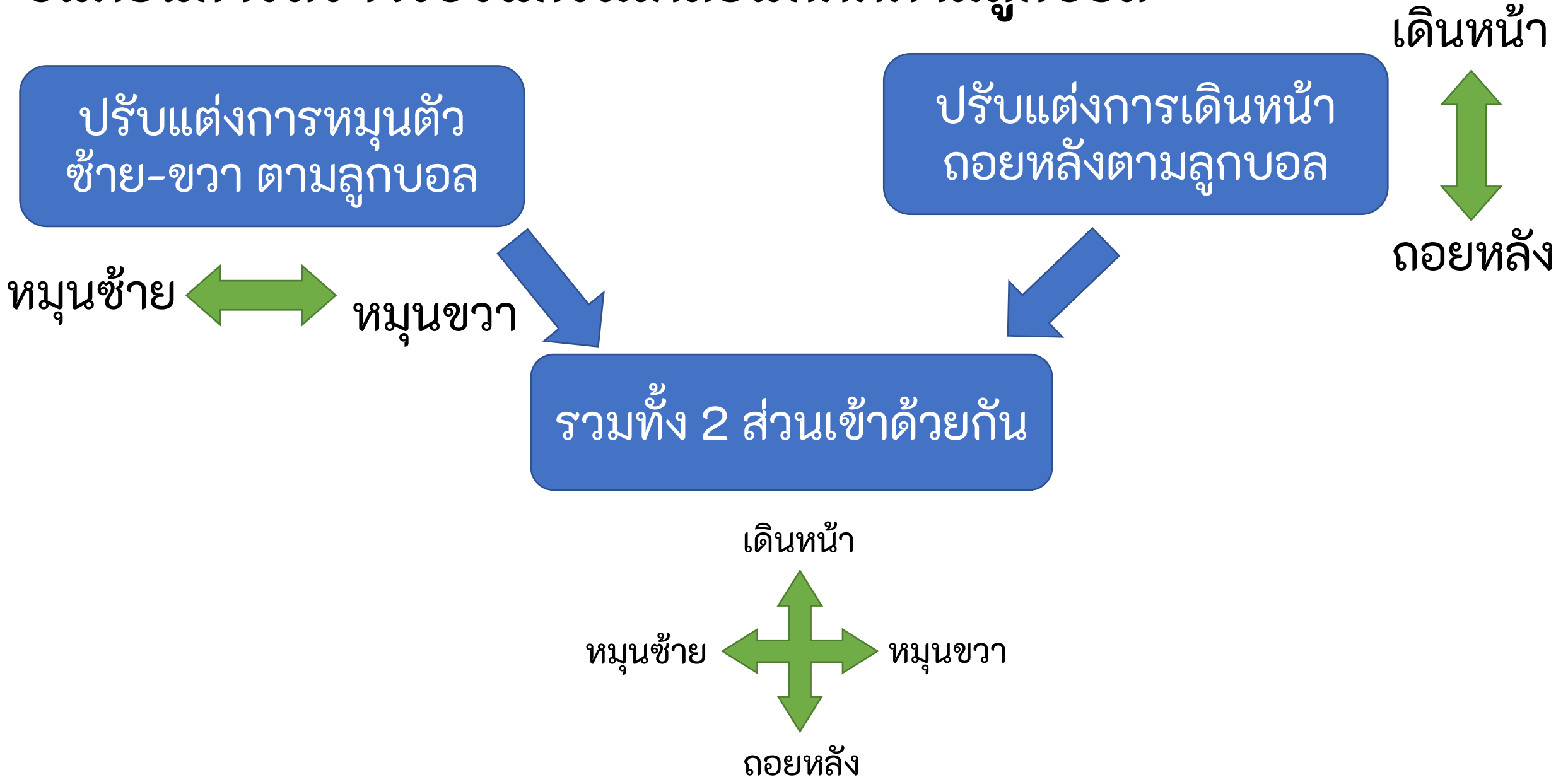
Kp : ช่วยทำหน้าที่ควบคุมทิศทางหุ้่นยนต์ให้เข้าใกล้สู่จุดที่ค่าผิดพลาดเท่ากับ 0

Ki : ช่วยให้หุ้่นยนต์เข้าใกล้สู่จุดที่ค่าผิดพลาดเท่ากับ 0 มากขึ้น

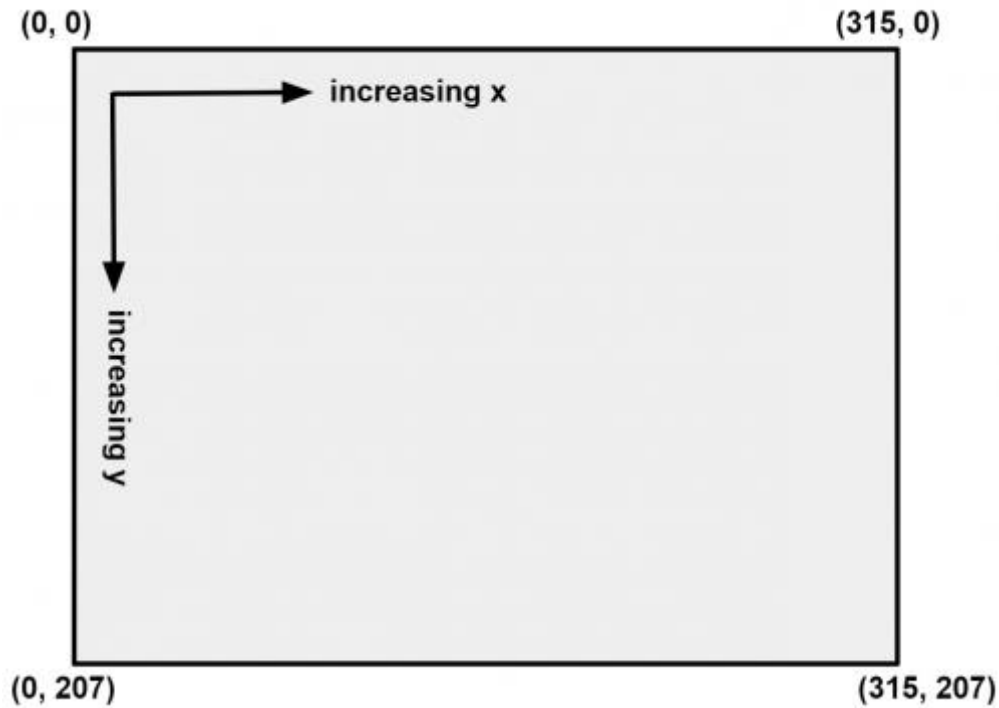
Kd : ช่วยลดการสละบัด ให้หุ้่นยนต์เข้าสู่จุดที่ค่าผิดพลาดเท่ากับ 0 ได้เร็ว

การปรับจูนค่า Kp, Ki, Kd จะขึ้นอยู่กับความเร็วของมอเตอร์, การตอบสนองของค่า Error กล่าวคือความเร็วของตัวประมวลผลหรือการตอบสนองของเซนเซอร์ที่ใช้งานนั่นเอง ในโปรแกรมตัวอย่างจะเห็นว่าอาจปรับแค่ค่า kp ก็เพียงพอที่จะทำให้หุ้่นยนต์เดินตามลูกบอลได้อย่างมีประสิทธิภาพแล้ว

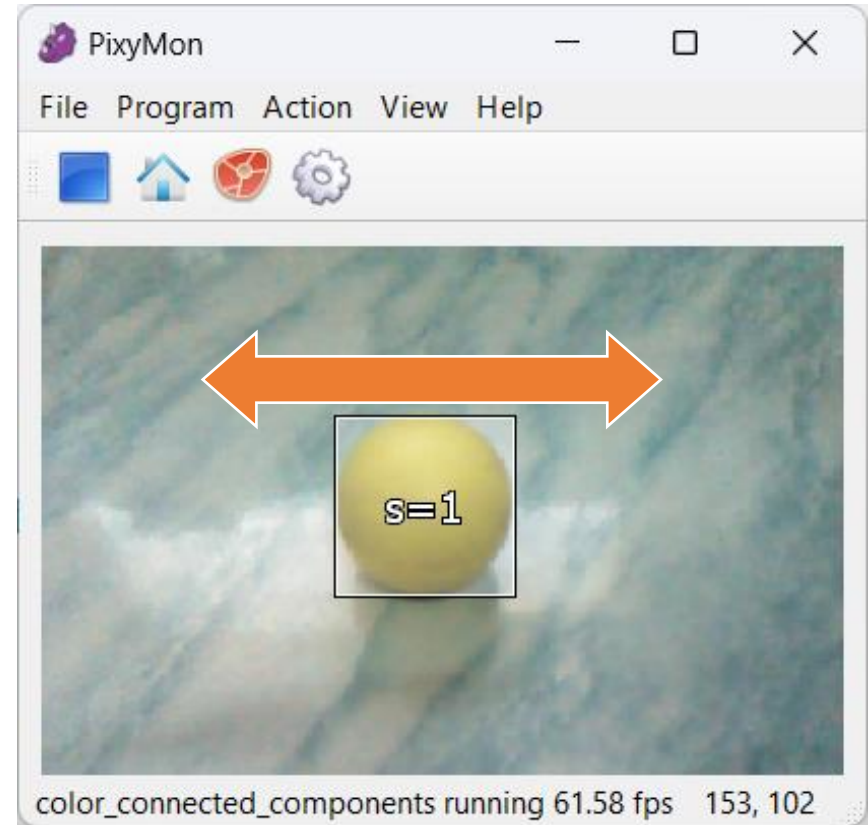
ขั้นตอนการสร้างโปรแกรมเคลื่อนที่ติดตามลูกบอล



ปรับแต่งการหมุนตัวซ้าย-ขวา ตามลูกบอล



ใช้แกน x มาพิจารณาการปรับทิศ



สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับทิศทางหุ่นยนต์หมุน ช่าย-ขวา

```
#include <POP32.h>
#include <POP32_Pixy2.h>
POP32_Pixy2 pixy;
#define MAX_Spd 60
#define head_Kp 0.1
#define head_Ki 0.0
#define head_Kd 0.0
float head_sp = 150;
float head_error,hl,hD,hPrevError,head_spd;
```

เรียกใช้ไลบรารี POP32 และ Pixy
ประกาศตัวแปรและกำหนดค่า

```
void setup(){
  pixy.init();
  waitAnykey_bmp();
}
```

เริ่มต้นใช้งาน Pixy และ รอกดปุ่ม

คำสั่ง

constrain(value,min,max)

เป็นคำสั่งที่ใช้กำหนดค่าให้อยู่ในช่วงที่เราต้องการ

ตัวอย่าง

```
int y = constrain(50,10,30)
```

y จะมีค่าเท่ากับ 30

P : สัดส่วนความผิดพลาด

$$\text{Error} = \text{Set_point} - \text{feedback}$$

```
float head_sp = 150;
```

กำหนดค่า Set point

```
int x = pixy.sigInfo[1][0].x  
head_error = head_sp - x
```

หาค่าความผิดพลาด

```
head_spd = (head_Kp * head_error)
```

เพิ่มอัตราการขยาย

I : สัดส่วนความผิดพลาดสะสม

$$\text{SumError} = \text{SumError} + \text{Error}$$

$$hl = hl + \text{head_error}$$

สะสมค่าความผิดพลาด

$$hl = \text{constrain}(hl, -\text{MAX_Spd}, \text{MAX_Spd})$$

จำกัดกรอบค่าความผิดพลาด ไม่ให้สูงเกินค่าความเร็วสูงสุด

$$\text{head_spd} = (hl * \text{head_Ki})$$

เพิ่มอัตราการขยาย

D : สัดส่วนความผิดพลาดปัจจุบันต่ออดีต

$$dError = Error - PrevError$$

$$hD = head_error - hPrevError$$

command1;

command2;

คำสั่งต่างๆที่เกี่ยวข้องในโปรแกรม

command3;

command4;

$$hPrevError = head_error$$

ผลต่างค่าความผิดพลาด
ปัจจุบันต่ออดีต

$$head_spd = (hD * head_Kd)$$

เพิ่มอัตราการขยาย

D : สัดส่วนความผิดพลาดปัจจุบันต่ออดีต

$$dError = Error - PrevError$$

$$head_spd = (head_Kp * head_error) + (hI * head_Ki) + (hD * head_Kd)$$

รวมทุกเทอมเข้าด้วยกัน

```
int leftSpeed = constrain(-head_spd, -MAX_Spd, MAX_Spd)
int rightSpeed = constrain(head_spd, -MAX_Spd, MAX_Spd)
```

คำสั่งปรับความเร็วมอเตอร์

```
motor(1, leftSpeed)
motor(2, rightSpeed)
```

ปรับทิศด้วยความเร็วด้านซ้ายให้ตรงกับหุ่นยนต์
และจำกัดค่าไม่ให้เกินความเร็วมอเตอร์ที่ต้องการ

สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับทิศหุ่นยนต์หมุน ช้าย-ขวา

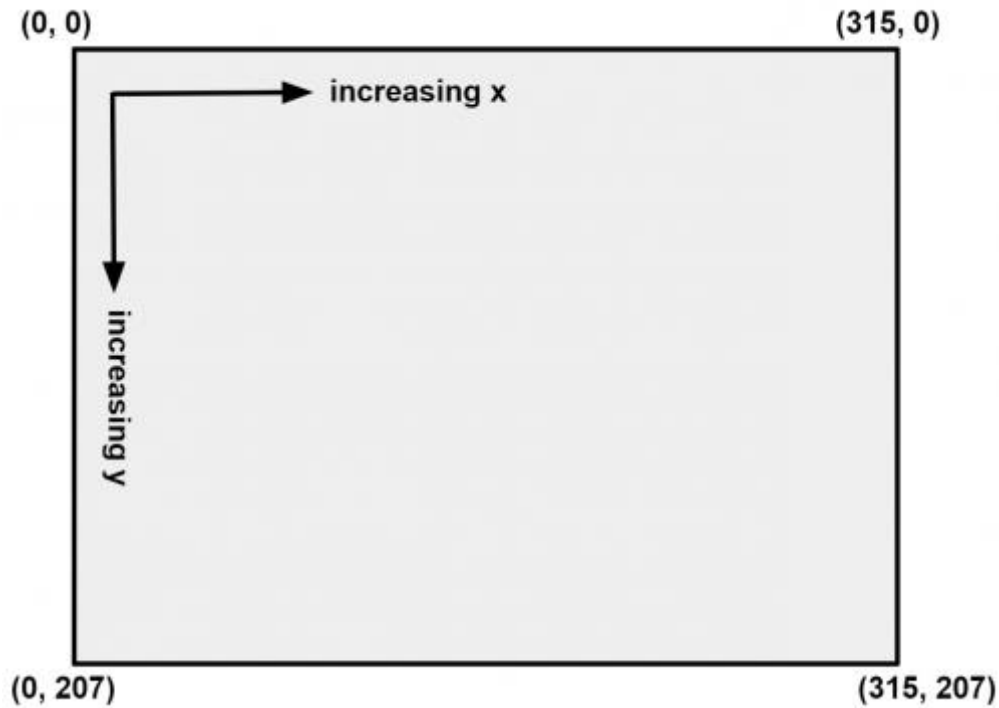
```
void loop() {  
  if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    int x = pixy.sigInfo[1][0].x;  
    head_error = head_sp - x;  
    hl = hl + head_error;  
    hl = constrain(hl, -MAX_Spd, MAX_Spd);  
    hD = head_error - hPrevError;  
    head_spd = (head_Kp * head_error) + (hl * head_Ki) + (hD * head_Kd);  
    int leftSpeed = constrain(-head_spd, -MAX_Spd, MAX_Spd);  
    int rightSpeed = constrain(head_spd, -MAX_Spd, MAX_Spd);  
    motor(1, leftSpeed); motor(2, rightSpeed);  
    hPrevError = head_error;  
  }  
}
```

ปรับแต่งค่าเพื่อให้หุ่นยนต์หมุนตัวซ้าย-ขวา ไปตามลูกบอล

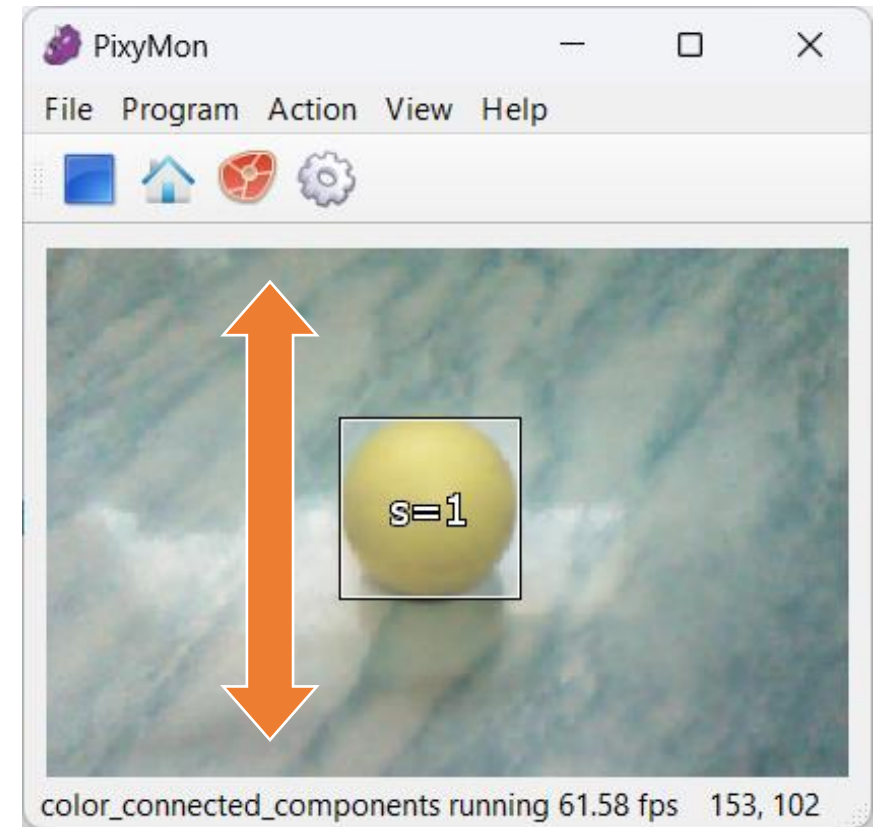
```
#define MAX_Spd 60 // กำหนดความเร็วสูงสุด  
#define head_Kp 0.1  
#define head_Ki 0.0  
#define head_Kd 0.0  
float head_sp = 150; // จุดกึ่งกลางกล้อง Pixy
```

เริ่มจากเพิ่มค่า Kp ก่อนเพื่อให้เข้าสู่จุด Set point ลูกบอลอยู่ตรงกลางหุ่นยนต์พอดี

ปรับแต่งการเดินทาง-ถอยหลังตามลูกบอล



ใช้แกน y มาพิจารณาการ



สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

```
#include <POP32.h>
#include <POP32_Pixy2.h>
POP32_Pixy2 pixy;
#define MAX_Spd 70
#define dis_Kp 0.1
#define dis_Ki 0.0
#define dis_Kd 0.0
float dis_sp = 160; // distance setpoint y
float dis_error, dI,dD , dPrevError, dis_spd;
```

เรียกใช้ไลบรารี POP32 และ Pixy
ประกาศตัวแปรและกำหนดค่า

```
void setup(){
  pixy.init();
  waitAnykey_bmp();
}
```

เริ่มต้นใช้งาน Pixy และ รอกดปุ่ม

สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

```
void loop() {  
  if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    int y = pixy.sigInfo[1][0].y;  
    dis_error = dis_sp - y;  
    dl = dl + dis_error;  
    dl = constrain(dl, -MAX_Spd, MAX_Spd);  
    dD = dis_error - dPrevError;  
    dis_spd = (dis_Kp * dis_error) + (dl * dis_Ki) + (dD * dis_Kd);  
    int leftSpeed = constrain(dis_spd, -MAX_Spd, MAX_Spd);  
    int rightSpeed = constrain(dis_spd, -MAX_Spd, MAX_Spd);  
    motor(1, leftSpeed); motor(2, rightSpeed);  
    dPrevError = dis_error;  
  }  
}
```

สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

```
#define MAX_Spd 70 // กำหนดความเร็วสูงสุด  
#define dis_Kp 0.1  
#define dis_Ki 0.0  
#define dis_Kd 0.0  
float dis_sp = 160; // จุดลูกบอลใกล้สุดที่ต้องการ
```

เริ่มจากเพิ่มค่า Kp ก่อนเพื่อให้เข้าสู่จุด Set point

รวมการเคลื่อนที่ในแต่ละแกนเข้าด้วยกัน

```
int leftSpeed = constrain(dis_spd - head_spd, -MAX_Spd, MAX_Spd);  
int rightSpeed = constrain(dis_spd + head_spd, -MAX_Spd, MAX_Spd);
```

โดยการนำค่า dis_spd และ head_spd มารวมกัน แล้วเก็บไว้ในตัวแปรกำหนดความเร็ว

ระบุเงื่อนไขเมื่อถึงเป้าหมาย

```
const int h_gap = 10;
const int d_gap = 10;
if ((abs(head_error) < h_gap) && (abs(dis_error) < d_gap)){
    motor(1, 0);
    motor(2, 0);
    beep();
}
```

โดยการนำค่า head_error และ dis_error มากำหนดช่วงที่ต้องการ ถ้าเข้าใกล้จุด Set Point โดยอยู่ในช่วงค่าผิดพลาดเท่ากับ 10 ทั้ง 2 ค่าถือว่าถึงเป้าหมายแล้ว